

UNIVERSITÀ DEGLI STUDI DI ROMA “TOR VERGATA”

Macroarea di Ingegneria
Corso di Laurea Magistrale in Ingegneria Informatica

Prodotto matrice densa per multivettore in MPI e CUDA

Progetto di fine corso

Sistemi di Calcolo Parallelo e Applicazioni
A.A. 2025/2026

Autori

Alessandro Vassallo
0366572

Andrea Galluzzi
0365026

Codice sorgente: <https://github.com/AleVassa00/parallel-computing-project>

Versione della relazione del 15 settembre 2026

Bozza di lavoro. I marcatori in rosso segnalano i punti che dipendono da campagne di misura non ancora concluse. Per la versione di consegna, commentare `\todovisibiletrue` in `preambolo.tex`.

Indice

1	Introduzione	1
1.1	Obiettivo	1
2	Il kernel di calcolo	2
2.1	Che cosa calcola il nucleo	2
2.1.1	Conteggio delle operazioni	2
2.2	Come vengono memorizzati i dati	3
2.3	Ordine dei cicli: schema A contro schema B	3
2.3.1	Gli schemi candidati	4
2.3.2	La quantificazione: intensità aritmetica	4
2.3.3	Un limite che non si può superare	5
2.3.4	Una proprietà che conta sull'hardware reale	5
2.4	Precisione	5
2.5	Generazione riproducibile dei dati	6
2.5.1	Il generatore	6
2.5.2	Le tre conseguenze, tutte usate dal progetto	6
2.5.3	Perché i valori hanno media nulla	6
2.6	L'interfaccia unica del kernel locale	7
2.6.1	Ciclo di vita del kernel	7
2.6.2	Contesto opaco del backend	8
2.6.3	Configurazione delle build	8
3	Parallelizzazione MPI	9
3.1	Che cosa significa parallelizzare in MPI	9
3.2	Perché si taglia A , e perché in due direzioni	10
3.2.1	Che cosa tagliare	10
3.2.2	Come tagliare: due possibilità monodimensionali	10
3.2.3	Il taglio bidimensionale	10
3.2.4	Perché i processi vengono organizzati a griglia	11
3.3	Decomposizione a griglia bidimensionale	11
3.3.1	I due sotto-comunicatori	12
3.3.2	Chi fa da root, e perché la scelta è verificata	12
3.4	Partizionamento degli indici	13
3.4.1	Le funzioni	13
3.5	Layout dei blocchi locali	13
3.5.1	Perché i conteggi delle collettive tornano	13
3.5.2	Chi contiene che cosa, e quando	14
3.5.3	Perché il multivettore non viene mai tagliato lungo k	14
3.6	Distribuzione iniziale dei dati	14
3.7	L'algoritmo distribuito	15

3.8	Modello di costo della comunicazione	16
4	Il nucleo di CPU	18
4.1	Tre backend, una sola idea	18
4.2	Il backend di riferimento	18
4.2.1	Perché k deve essere noto al compilatore	18
4.2.2	Il fallback generico, e perché la traccia lo richiede	19
4.2.3	Il microbenchmark che isola la specializzazione	19
4.3	Perché $KB = 32$	20
4.4	Ciò che il compilatore vede	20
4.5	Il primo limite: la rilettura di X	21
4.6	Secondo backend: X mattonellata in cache	21
4.7	Terzo backend: register blocking sulle righe	22
5	I backend CUDA	23
5.1	Struttura comune	23
5.2	<code>cuda_naive</code> : la baseline	25
5.3	<code>cuda_warp</code> : un warp per riga	25
5.3.1	Un secondo layout per X : <code>X_LAYOUT=column</code>	25
5.4	<code>cuda_warp_smem</code> : la fetta di X in shared memory	26
5.4.1	Il conflitto sui banchi e il padding a $k + 1$	27
5.4.2	Che cosa il tile cambia davvero, e che cosa no	27
5.4.3	Perché non ci sono uscite anticipate	28
5.5	<code>cuda_warp_tiled</code> : meno accumulatori per lane	28
5.6	<code>cublas</code> : il riferimento esterno	28
5.6.1	Riga contro colonna: nessuna trasposizione	29
5.6.2	L'attesa è che vada male	29
5.7	Più rank MPI sulla stessa GPU	29
6	Modello roofline	31
6.1	Roofline della GPU	31
6.1.1	Che cosa il modello non dice	33
6.2	Roofline della CPU	33
7	Metodologia sperimentale	34
7.1	Ambiente sperimentale	34
7.2	Protocollo di misura	35
7.3	Definizione della metrica di prestazione	36
7.4	Costi esclusi dalla metrica ufficiale	38
7.5	Ripetizioni e stabilità statistica	38
7.6	Disegno delle campagne sperimentali	39
7.6.1	C1 — Dimensione del problema e rapporto $M:N$	40
7.6.2	C2 — Sweep su k e confronto dei backend	40
7.6.3	C4 — Strong scaling	40
7.6.4	C5 — Weak scaling	41
7.6.5	C6 — Forma della griglia MPI	41
7.6.6	C7 — Precisione numerica	42
8	Implementazione seriale, correttezza e validazione	44
8.1	L'oracolo seriale	44
8.2	Una tolleranza che dipende da N	44
8.3	Il test delle funzioni indice	45

8.4	La matrice di collaudo	46
9	Risultati	47
9.1	Stato della raccolta dati	47
9.2	C3 — baseline di CPU	47
9.2.1	Lettura: due regimi separati da una soglia di cache	48
9.2.2	Conseguenza pratica	49
9.2.3	Specializzazione contro fallback	50
9.3	C2 — confronto fra backend al variare di k	50
9.3.1	Sweep sul padding della shared memory	51
9.3.2	Sweep sulla dimensione del blocco	52
9.4	C1 — taglia e rapporto $M:N$	52
9.5	C6 — perché la griglia è bidimensionale	53
9.6	C4 — strong scaling	53
9.7	C5 — weak scaling	54
9.8	C7.1 — quanto rende la singola precisione	55
9.9	C7.2 — perché: registri, spill e shared memory	56
9.10	Il costo dei dati: trasferimenti e preprocessing	56
9.11	Sintesi	57
10	Conclusioni	58
10.1	Che cosa è stato realizzato	58
10.2	Che cosa dicono i dati finora	58
10.3	Lavori futuri	59
A	Interfaccia a riga di comando e formato dei dati	60
A.1	Opzioni del driver	60
A.2	Colonne del CSV	60
A.3	Opzioni di build	61
B	Riproduzione dei risultati	63
B.1	Build	63
B.2	Validazione	63
B.3	Esecuzione singola	64
B.4	Campagne	64
B.5	Unione dei CSV di una campagna	64
B.6	Note operative sul server	64
B.7	Compilazione di questa relazione	65

Capitolo 1

Introduzione

1.1 Obiettivo

Il progetto realizza un nucleo di calcolo per il prodotto fra una matrice densa e un multivettore,

$$Y \leftarrow AX, \quad A \in \mathbb{R}^{M \times N}, \quad X \in \mathbb{R}^{N \times k}, \quad Y \in \mathbb{R}^{M \times k}, \quad (1.1)$$

con k *piccolo*: il multivettore è una matrice densa alta e strettissima. Il codice funziona per k generico ed è collaudato sui cinque valori richiesti dalla traccia, $k \in \{3, 6, 8, 20, 32\}$.

La parallelizzazione è ibrida a due livelli:

- **MPI (Message Passing Interface)** costituisce il livello esterno, responsabile della distribuzione dei dati su una griglia di processi bidimensionale e della comunicazione fra processi
- la parallelizzazione interna al singolo processo è affidata a **CUDA (Compute Unified Device Architecture)**, che costituisce il backend di accelerazione su GPU NVIDIA.

I due livelli non sono alternativi ma sovrapposti: ogni processo MPI possiede un blocco di A e delega il calcolo del proprio contributo parziale a un *backend* locale, che può essere un kernel CUDA oppure un nucleo di CPU.

Capitolo 2

Il kernel di calcolo

Questo capitolo fissa tre cose che il resto della relazione dà per acquisite: che cosa il nucleo calcola e come se ne contano le operazioni, in che ordine conviene scandire i tre cicli, e da dove vengono i dati. Sono scelte fatte una volta sola, prima di qualunque backend: i capitoli su MPI, CPU e CUDA le ereditano senza rimetterle in discussione.

2.1 Che cosa calcola il nucleo

Il nucleo calcola $Y \leftarrow AX$, con A densa $M \times N$ e X multivettore $N \times k$. Elemento per elemento:

$$Y[i][c] = \sum_{j=0}^{N-1} A[i][j] X[j][c], \quad 0 \leq i < M, \quad 0 \leq c < k. \quad (2.1)$$

La traccia fissa i gradi di libertà: M ed N sono scelte dall'utente a runtime, k deve funzionare per valore *generico* e il collaudo deve coprire almeno $k = 3, 6, 8, 20, 32$. Il progetto rispetta entrambe le richieste, ma con una distinzione che conviene anticipare subito: sui cinque k del collaudo il nucleo di CPU usa funzioni specializzate a tempo di compilazione, mentre ogni altro k passa da un percorso generico con prestazioni diverse (capitolo 4).

2.1.1 Conteggio delle operazioni

Il conteggio si costruisce in due passi.

Passo 1 - un singolo elemento. Ogni $Y[i][c]$ è una somma di N prodotti: servono N moltiplicazioni e $N - 1$ addizioni, ma la convenzione standard, e quella imposta dalla traccia, arrotonda a 2 FLOP per termine.

Passo 2 - tutta la matrice. Gli elementi di Y sono $M \cdot k$.

Quindi:

$$\text{FLOP}(M, N, k) = 2 M N k, \quad \text{GFLOPS} = \frac{2 M N k}{T \cdot 10^9}. \quad (2.2)$$

Su questa formula la traccia impone tre vincoli, che vale la pena enunciare qui perché condizionano tutta la metodologia di misura:

1. T è il tempo **medio per invocazione**, ottenuto ripetendo il nucleo più volte sulla stessa matrice.

2. In T entra **solo il calcolo**: preprocessing, I/O ed eventuali trasferimenti da e verso la scheda CUDA restano fuori. La traccia consente però di misurarli e discuterli a parte, ed è quello che il progetto fa. Che cosa esattamente venga cronometrato, e con quale orologio, è trattato in ??.
3. M ed N sono le dimensioni della matrice **globale**, anche quando l'esecuzione è distribuita. È proprio questo che rende leggibile lo speedup: il lavoro totale non cambia con il numero di processi, cambia solo T .

2.2 Come vengono memorizzati i dati

Tutte e tre le matrici sono **row-major**: *gli elementi di una stessa riga sono adiacenti*.

La matrice A ha inoltre una propria *leading dimension* lda , cioè la distanza in elementi fra l'inizio di una riga e l'inizio della successiva nella rappresentazione in memoria. La leading dimension è stata introdotta per permettere *padding* fra righe consecutive e per ricevere direttamente blocchi non contigui mediante datatype MPI derivati. In realtà, le matrici X e Y la leading dimension è *sempre* fissata a k , che è il numero di colonne, e quindi le righe sono compatte. La leading dimension è sfruttata:

$$A[i][j] = A_loc[i \cdot \text{lda} + j], \quad X[j][c] = X_loc[j \cdot \text{ldx} + c], \quad Y[i][c] = Y_loc[i \cdot \text{ldy} + c].$$

Quando lda coincide con il numero di colonne il buffer è compatto; quando è maggiore, fra una riga e l'altra restano delle celle inutilizzate, per l'appunto il *padding*.

Perché lda è separata da n_loc . Il blocco che un processo possiede è una sottomatrice di A , quindi **non è contiguo nella matrice globale**: le sue righe sono distanti N elementi l'una dall'altra. La distribuzione usa per questo un datatype derivato `MPI_Type_vector`, con stride N dal lato del mittente e stride lda dal lato del ricevente, così il blocco viene depositato direttamente nel buffer locale senza copie intermedie (capitolo 3). Tenere lda come parametro indipendente aggiunge a questo la possibilità di avere padding fra le righe locali.

Perché ldx e ldy invece sono fissate a k . L'interfaccia del kernel (??) le espone e ogni backend le onora, ma il driver distribuito impone $\text{ldx} = \text{ldy} = k$, cioè buffer compatti. La ragione è nelle collettive: il `MPI_Bcast` di X e il `MPI_Reduce` di Y lavorano con conteggi contigui, e descrivere uno stride richiederebbe di costruire un datatype derivato a ogni invocazione, *dentro* la regione cronometrata. A invece non attraversa nessuna collettiva, ed è per questo che è l'unica delle tre a potersi permettere il padding.

Il fatto che le k colonne di X siano adiacenti in memoria non è un dettaglio di comodo: è la proprietà su cui si regge tutta la sezione 2.3.

Ipotesi:

Un backend CUDA prevede in alternativa una disposizione **column-major** di X , ma come opzione di build e con effetti discussi al momento giusto (??); il default, e tutto ciò che segue, è row-major.

2.3 Ordine dei cicli: schema A contro schema B

La eq. (2.1) contiene tre indici, i , j e c . Indipendentemente dall'ordine in cui viene scelto di scandire gli indici, l'operazione produce lo stesso risultato, ma il diverso ordine con il quale vengono scanditi non danno affatto le stesse prestazioni. È il primo punto in cui il progetto si allontana da una trascrizione letterale della definizione.

2.3.1 Gli schemi candidati

I due candidati sono:

Schema A ($i \rightarrow j \rightarrow c$)

```
for i:
    acc[0..k) = 0
    for j:
        a = A[i][j]
        for c:
            acc[c] += a * X[j][c]
Y[i][0..k) = acc[0..k)
```

Schema B ($i \rightarrow c \rightarrow j$)

```
for i:
    for c:
        s = 0
        for j:
            s += A[i][j] * X[j][c]
        Y[i][c] = s
```

Lo schema B è la trascrizione letterale della definizione, ed è esattamente l'esecuzione di k prodotti matrice-vettore indipendenti, uno per colonna. Lo schema A scambia i due cicli interni, cambiando anche il “ruolo” della matrice A .

Si segua una singola riga i .

Schema B. Il ciclo su c sta fuori e quello su j dentro, quindi per ogni colonna c il ciclo interno percorre l'intera riga $A[i][0 \dots N-1]$. Finita una colonna si ricomincia da capo con la successiva, sulle stesse celle. La riga viene dunque letta k volte, per un totale di kN letture di A a fronte delle N celle che la riga contiene.

Schema A. Il ciclo su j sta fuori e quello su c dentro. Il valore $a = A[i][j]$ viene letto una volta sola e consumato subito da tutti e k gli accumulatori, con gli aggiornamenti $\text{acc}[c] += a \cdot X[j][c]$ per $c = 0 \dots k-1$; poi si passa a $j+1$ e quel valore di A non serve più. La riga viene letta una volta sola, per un totale di N letture. Il riuso di A vale quindi esattamente k , e vale per costruzione: non dipende dalla taglia del problema né da condizioni sul valore di k .

Perché il risparmio sia reale gli accumulatori devono stare in registro, e perché il compilatore possa tenerceli deve conoscerne il numero a tempo di compilazione: è il motivo delle specializzazioni su k descritte in sezione 4.2.1.

Si potrebbe obiettare che le riletture dello schema B le assorbe la cache, e in parte è così, ma solo finché la dimensione della riga è contenibile nella cache. Una riga di A occupa Ns byte, dove s è la dimensione in byte di ogni elemento della matrice e N è la dimensione che la traccia chiede di far crescere: il caso interessante è proprio quello in cui la riga eccede i livelli di cache vicini al calcolo o, sulla GPU, quello in cui la riga eccede la shared memory di un blocco. Il modello che segue conta perciò il traffico verso la memoria principale, che è il livello che finisce per determinare il tempo.

2.3.2 La quantificazione: intensità aritmetica

L'*intensità aritmetica* I è il rapporto fra i FLOP eseguiti e i byte che attraversano il livello di memoria considerato. Detta s la dimensione in byte dello scalare, e contando il solo traffico su A come giustificato in ??:

$$I_A = \frac{2MNk}{MNs} = \frac{2k}{s}, \quad I_B = \frac{2MNk}{kMNs} = \frac{2}{s}. \quad (2.3)$$

Il numeratore è lo stesso: i FLOP da fare sono fissati dal problema. Cambia il denominatore, perché lo schema B legge gli MN elementi di A una volta per ciascuna delle k colonne.

In FP64 ($s = 8$) questo significa $I_A = k/4$ contro $I_B = 1/4$ FLOP/B: lo schema A guadagna esattamente un fattore k , e a $k = 32$ porta l'intensità da 0,25 a 8 FLOP/B.

È questo salto che sposta il nucleo da un regime saldamente memory-bound a un regime in cui, sulla GPU in doppia precisione, il limite può diventare il calcolo. La discussione quantitativa, con i picchi reali della scheda, è nel capitolo 6.

2.3.3 Un limite che non si può superare

$I_A = 2k/s$ è il **massimo** ottenibile: gli MN elementi di A vanno letti almeno una volta e i FLOP sono fissati, quindi nessuna riorganizzazione ulteriore può migliorare il rapporto. Tutto il lavoro di ottimizzazione successivo, su CPU come su GPU, riguarda allora il traffico su X e la capacità di tenere i k accumulatori in registro, non l'intensità rispetto ad A , che è già al suo limite.

Con un'eccezione, che la traccia rende rilevante chiedendo k generico. Tenere k accumulatori in registro è possibile finché k è abbastanza piccolo: oltre una certa ampiezza il nucleo di CPU blocca le colonne a gruppi di $KB = 32$ e rilegge A una volta per gruppo, cosicché in generale

$$I_A(k) = \frac{2k}{s \cdot \lceil k/KB \rceil}.$$

Per i cinque k del collaudo il ciclo sui blocchi è degenere, A viene letta una volta sola e si ricade nella eq. (2.3); per $k = 64$ i passaggi sono due e l'intensità si ferma al valore che aveva a $k = 32$. Il dettaglio, con il microbenchmark che lo misura, è in sezione 4.2.1.

2.3.4 Una proprietà che conta sull'hardware reale

Nello schema A entrambi i flussi di lettura sono a **passo unitario**. La riga di A è contigua per costruzione e la riga di X lo è perché X è row-major con le k colonne adiacenti (sezione 3.5). Nessuno dei due accessi ha stride, quindi i prefetcher hardware lavorano nelle condizioni migliori e, sulla GPU, gli accessi delle lane di uno stesso warp sono coalescibili. Lo schema B, per ottenere la stessa proprietà sull'accesso a X , richiederebbe X column-major, cioè un layout diverso da quello che serve altrove.

2.4 Precisione

Tutto il codice usa un unico tipo `scalar_t`, definito in `src/common/scalar.h`, che vale `double` per default e `float` compilando con `PREC=float` (il Makefile traduce la variabile nella macro `-DUSE_FLOAT`). Anche il tipo MPI segue lo scalare, attraverso la macro `SCALAR_MPI_TYPE`, così le collettive non vanno toccate.

Il confronto FP64/FP32 è quindi un flag di compilazione e non una seconda copia dei sorgenti: le due build eseguono esattamente lo stesso codice, che è la sola condizione in cui il confronto misura la precisione e non due implementazioni diverse.

Effetto sul modello roofline. La scelta della precisione non è neutra, e agisce in **due modi opposti**:

- dimezzando s la eq. (2.3) *raddoppia* l'intensità aritmetica, da $k/4$ a $k/2$. Da solo, questo avvicinerebbe il nucleo al regime compute-bound;
- ma il punto di crossover del roofline vale P_{picco}/B , e su una GPU Turing il picco FP32 è 32 volte il picco FP64. Il crossover si sposta quindi a destra di un fattore 32.

Il secondo effetto è 16 volte più forte del primo e prevale nettamente: in singola precisione il nucleo finisce **saldamente memory-bound** su tutto l'intervallo di k della traccia, mentre in doppia precisione è compute-bound già da $k = 6$. I due regimi sono qualitativamente diversi,

ed è questo il vero argomento con cui il progetto discute la precisione: la doppia non costa il fattore 2 del traffico, costa il fattore 32 del picco. I numeri sono in sezione 6.1.

Effetto sulla validazione. `scalar.h` deriva dal tipo anche `SCALAR_EPS` e la tolleranza del confronto con l'oracolo seriale, che non è una costante ma cresce con la lunghezza della riduzione. Il modello e la costante misurata sono discussi in ??.

2.5 Generazione riproducibile dei dati

Le matrici di test sono generate, non lette da file. La traccia lo consente esplicitamente, elencando la generazione fra le strategie di collaudo, e in cambio chiede che sia **ripetibile**.

2.5.1 Il generatore

Il generatore è `splitmix64`, un mixer a 64 bit **senza stato**. La differenza rispetto a un generatore pseudocasuale ordinario è tutta qui: un generatore con stato produce una *sequenza*, e per ottenere l'elemento in posizione p bisogna averlo fatto avanzare p volte. Un mixer stateless va da (seed, chiave) al valore in un passo solo. La chiave scelta è l'**indice lineare globale** dell'elemento:

$$A[i][j] = g(\text{seed}, \Sigma_A, i \cdot N + j), \quad X[j][c] = g(\text{seed}, \Sigma_X, j \cdot k + c), \quad (2.4)$$

dove Σ_A e Σ_X sono due identificatori di *stream* distinti, che servono a evitare che A e X abbiano lo stesso pattern di valori. Il seed viene mescolato prima con lo stream e poi con la chiave: due passaggi, perché uno solo lascerebbe correlati i valori di chiavi vicine. Dei 64 bit prodotti si usano i 53 alti, che danno un `double` esatto in $[0, 1)$, riscalato poi in $[-1, 1)$.

Il valore di un elemento dipende dunque *solo* dalla sua posizione globale: mai dal processo che lo calcola, mai dall'ordine in cui viene generato, mai dalla forma della griglia.

2.5.2 Le tre conseguenze, tutte usate dal progetto

1. ogni processo può generare **direttamente** il proprio blocco locale, senza che una copia globale della matrice esista mai in memoria. È ciò che permette di misurare taglie che su un solo nodo non starebbero;
2. la stessa matrice globale si ottiene con *qualunque* griglia di processi e con qualunque modalità di inizializzazione. Due configurazioni diverse sono quindi confrontabili elemento per elemento;
3. l'esecuzione seriale di ??, usata come oracolo, può ricostruire da sé la porzione che gli serve, senza ricevere dati dagli altri processi.

2.5.3 Perché i valori hanno media nulla

I valori sono distribuiti in $[-1, 1)$ a media nulla, e questa non è una scelta estetica. Ogni elemento di Y è una riduzione su N termini, e l'errore di arrotondamento di una riduzione dipende da come si compongono i segni: con termini a media nulla gli arrotondamenti hanno segno indipendente e l'errore cresce come $\varepsilon\sqrt{N}$, non come il pessimistico εN del caso in cui tutti i termini abbiano lo stesso segno. È questa proprietà dei dati che rende possibile una tolleranza di validazione stretta invece di una soglia fissa e generosa (??).

Una nota utile per il confronto fra precisioni: il generatore calcola sempre in `double` e converte in `scalar_t` solo alla fine. Le build FP64 e FP32 lavorano quindi sugli stessi dati a meno del solo arrotondamento della conversione, ed è la condizione che rende confrontabili i loro tempi.

2.6 L'interfaccia unica del kernel locale

Ogni backend implementa tre operazioni fondamentali per la gestione e l'esecuzione del kernel locale, oltre ad alcune funzioni ausiliarie utilizzate per la raccolta delle metriche sperimentali.

2.6.1 Ciclo di vita del kernel

```
/* esecuzione */
local_gemm_t *local_gemm_create(int m, int n, int k,
                                const scalar_t *A_loc,
                                int lda, int ldx, int ldy);
void local_gemm(local_gemm_t *ctx,
                const scalar_t *RESTRICT X, int ldx,
                scalar_t *RESTRICT Y, int ldy);

void local_gemm_destroy(local_gemm_t *ctx);
```

Listato 2.1: Interfaccia di src/kernel/kernel.h.

L'interfaccia segue un ciclo di vita composto da tre fasi rappresentate da altrettante funzioni che definiscono il ciclo di vita del kernel locale:

- `local_gemm_create` ne inizializza il contesto,
- `local_gemm` esegue il calcolo e
- `local_gemm_destroy` rilascia le risorse associate.

Inoltre l'interfaccia espone alcune funzioni di introspezione utilizzate dal driver di benchmark per raccogliere separatamente tempi di calcolo, trasferimenti, costi di inizializzazione e parametri di esecuzione del backend. Tali metriche vengono descritte nella sezione dedicata alla metodologia sperimentale.

La separazione tra `local_gemm_create`, `local_gemm` e `local_gemm_destroy` deriva dal diverso ciclo di vita dei dati e delle risorse coinvolte nel calcolo.

La matrice A rimane invariata durante tutte le ripetizioni, mentre X e Y sono coinvolte in ogni esecuzione del prodotto locale. Questa differenza permette di separare le operazioni eseguite una sola volta da quelle necessarie a ogni invocazione.

1. **Inizializzazione (`local_gemm_create`).** Viene creato il contesto del backend e vengono predisposte le risorse riutilizzabili durante le successive esecuzioni.

Nei backend CUDA vengono allocati i buffer sul device e A viene trasferita una sola volta nella memoria della GPU.

2. **Esecuzione (`local_gemm`).** Questa fase viene ripetuta per ogni invocazione del prodotto locale. La matrice X , precedentemente distribuita tra i processi tramite `MPI_Bcast`, viene utilizzata insieme al blocco locale di A per produrre il contributo locale a Y .

Nei backend CUDA vengono riutilizzati i buffer allocati durante l'inizializzazione: la matrice X viene trasferita sul device, viene eseguito il kernel e il risultato Y viene riportato in memoria host.

I contributi prodotti dai processi vengono successivamente combinati tramite `MPI_Reduce`.

3. **Finalizzazione (`local_gemm_destroy`).** Al termine delle ripetizioni vengono rilasciate le risorse associate al contesto, comprese le eventuali allocazioni sulla GPU.

Questa organizzazione evita di ripetere a ogni invocazione le operazioni associate ad A e separa il costo di inizializzazione dal costo delle successive esecuzioni del prodotto locale.

2.6.2 Contesto opaco del backend

Lo stato specifico di ciascun backend è mantenuto all'interno del tipo opaco `local_gemm_t`. La struttura concreta è definita separatamente da ogni implementazione e può quindi contenere informazioni differenti senza modificare l'interfaccia comune.

Ad esempio, un backend CPU può memorizzare le dimensioni del problema e il riferimento ad A , mentre un backend CUDA mantiene anche i puntatori alla memoria del device e le risorse necessarie all'esecuzione.

2.6.3 Configurazione delle build

Le principali scelte sperimentali, tra cui backend di calcolo, precisione numerica e alcuni parametri dei kernel CUDA, vengono selezionate a tempo di compilazione tramite il `Makefile`.

Le principali varianti di configurazione utilizzate nelle campagne sperimentali vengono associate a binari distinti, evitando che build differenti si sovrascrivano e rendendo immediatamente identificabile la configurazione utilizzata.

Capitolo 3

Parallelizzazione MPI

Fino a questo punto il prodotto è stato descritto come se avvenisse dentro un unico spazio di memoria: i cicli di sezione 2.3 leggono A e X e scrivono Y come se tutte e tre fossero lì, a disposizione. Questo capitolo rimuove quell'ipotesi. Il problema diventa far collaborare più processi che non condividono niente, e le scelte che ne derivano condizionano tutto il resto della relazione: la forma dei blocchi, che cosa viene cronometrato, e perfino quali configurazioni ha senso misurare.

3.1 Che cosa significa parallelizzare in MPI

MPI (*Message Passing Interface*) è uno standard per programmare macchine a **memoria distribuita**. Il modello di esecuzione è il seguente: lo stesso programma viene avviato in P copie, ciascuna un processo del sistema operativo a sé stante, con il proprio spazio di indirizzi **privato**. Nessun processo può leggere o scrivere una variabile di un altro. L'unico modo perché un dato passi da un processo all'altro è che qualcuno lo invii esplicitamente, con una chiamata di libreria, e qualcun altro lo riceva.

È una differenza sostanziale rispetto a un modello a memoria condivisa, dove i thread vedono gli stessi indirizzi e la parallelizzazione consiste essenzialmente nel ripartire le iterazioni di un ciclo. Qui ripartire le iterazioni non basta, e non è nemmeno la prima cosa da decidere: prima bisogna stabilire **chi possiede quale dato**. Da questa scelta discende tutto il resto, perché un processo può calcolare solo ciò per cui possiede gli ingredienti, e ogni ingrediente mancante è un messaggio da far viaggiare.

La parallelizzazione del nucleo si articola quindi in tre domande, in quest'ordine:

1. **decomposizione** - come si tagliano A , X e Y fra i processi;
 2. **località** - quale parte del risultato ciascun processo riesce a calcolare con i soli dati che possiede;
 3. **comunicazione** - che cosa manca, e in quale direzione deve viaggiare perché il risultato locale diventi il risultato giusto.
- Il **rank** è l'identificatore intero di un processo all'interno di un gruppo: è ciò che permette a un programma unico di comportarsi diversamente su processi diversi.
 - Un **comunicatore** è un gruppo di processi assieme a un contesto di comunicazione isolato. Ogni operazione MPI avviene *dentro* un comunicatore, e i rank hanno senso solo rispetto a quello: `MPI_COMM_WORLD` è il comunicatore predefinito che contiene tutti i processi, ma se ne possono costruire altri più piccoli, ed è esattamente ciò che il progetto fa.

- Un'operazione **collettiva** è un'operazione che tutti i processi di un comunicatore devono chiamare. Il progetto ne usa due: il *broadcast*, in cui un processo possiede un dato e al termine lo possiedono tutti; e la *reduce*, in cui ogni processo possiede un array della stessa forma, gli array vengono combinati elemento per elemento con un operatore associativo — qui la somma — e il risultato finisce su un solo processo.
- Il **root** è quel processo designato: la sorgente nel broadcast, il destinatario nella reduce.

3.2 Perché si taglia A , e perché in due direzioni

3.2.1 Che cosa tagliare

La prima decisione riguarda quale delle tre matrici vada distribuita. La risposta viene da ???: A ha MN elementi, X e Y ne hanno rispettivamente Nk e Mk , e con k piccolo per definizione A è più grande delle altre due di un fattore che cresce con la taglia del problema.

Replicare A su ogni processo sarebbe quindi la scelta peggiore possibile, e per due ragioni indipendenti: ogni nodo dovrebbe contenerla per intero, mettendo un tetto alla taglia massima trattabile a prescindere da quanti processi si usino; e ogni processo la rileggerebbe tutta, quando è proprio il traffico su A a determinare il tempo. A va dunque **tagliata**. X e Y , che sono piccole, possono invece permettersi di essere replicate lungo un asse, e vedremo che è esattamente ciò che conviene fare.

3.2.2 Come tagliare: due possibilità monodimensionali

Tagliare A significa assegnare a ogni processo una sottomatrice. Le decomposizioni più semplici la tagliano lungo una sola dimensione, e la eq. (2.1) dice subito che cosa comportano.

Taglio a bande di righe. Ogni processo possiede righe *interi* di A . Per calcolare $Y[i][c]$ serve tutta la riga i di A e tutte le righe di X : un processo che possiede righe intere possiede quindi tutti i fattori del primo tipo, ma gli serve X per intero. Il vantaggio è che il risultato che calcola è già *definitivo*: nessuna somma finale fra processi. Il costo è che X deve essere replicata su tutti i P processi.

Taglio a bande di colonne. Ogni processo possiede colonne di A , cioè, per ciascuna riga, solo una parte degli addendi della sommatoria. Gli basta quindi la porzione di X corrispondente alle proprie colonne — X viene tagliata e non replicata — ma ciò che produce è un risultato **parziale**, che riguarda tutte le righe di Y e va sommato con i parziali di tutti gli altri. La somma finale coinvolge P processi su un oggetto della taglia piena di Y .

Le due alternative sono speculari: la prima paga la replicazione di X su tutti, la seconda la riduzione di Y su tutti. In entrambe la comunicazione coinvolge sempre l'intero insieme dei processi, e questo è il difetto strutturale: aumentare P non rende i messaggi più piccoli, li rende soltanto più numerosi.

3.2.3 Il taglio bidimensionale

La terza possibilità taglia A in entrambe le direzioni: ogni processo possiede un *rettangolo* della matrice. Le due necessità di prima compaiono ora entrambe — servono le righe di X corrispondenti alle proprie colonne di A , e il proprio contributo a Y è parziale e va sommato — ma ciascuna coinvolge soltanto un **sottoinsieme** dei processi: quelli che condividono le stesse colonne di A per la prima, quelli che condividono le stesse righe per la seconda. Nessuna delle due comunicazioni è più globale.

È questa la decomposizione richiesta dalla traccia, ed è quella che il progetto implementa. Che sia effettivamente superiore alle due monodimensionali non è però qualcosa da accettare sulla fiducia: è una proprietà quantificabile, e la sezione 3.8 la dimostra con un modello di costo, mostrando che le due decomposizioni 1D ne sono i casi degeneri. Il confronto viene poi misurato direttamente in capitolo 9.

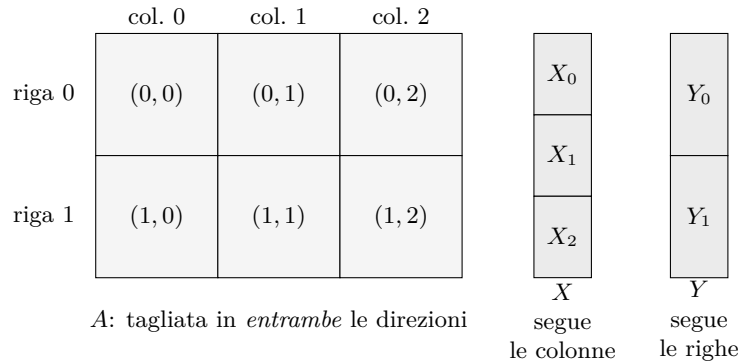


Figura 3.1: Decomposizione dei dati su una griglia 2×3 . Ogni processo possiede un *rettangolo* di A , individuato dalle sue coordinate nella griglia. Il multivettore X è tagliato per bande di righe, una per ciascuna banda di colonne di A ; Y per bande di righe, una per ciascuna banda di righe di A . Nessuna delle due è mai tagliata lungo k .

3.2.4 Perché i processi vengono organizzati a griglia

Resta un ultimo passaggio, che è di comodità ma non solo. Se il taglio del dato è bidimensionale, conviene che lo sia anche il modo in cui si identificano i processi: un processo individuato da una coppia di coordinate (r, c) invece che da un intero rende immediate le due relazioni che servono di continuo. «I processi che possiedono le stesse colonne di A » diventa «i processi della stessa colonna della griglia», e non un calcolo aritmetico sui rank da rifare ogni volta.

MPI offre questa struttura nativamente, sotto il nome di **topologia virtuale cartesiana**. È virtuale nel senso che non descrive come i nodi sono fisicamente cablati: descrive come il programma vuole *pensare* all'insieme dei propri processi, lasciando però all'implementazione la possibilità di sfruttare quell'informazione per disporli meglio sulla macchina reale.

3.3 Decomposizione a griglia bidimensionale

I P processi vengono organizzati in una griglia cartesiana $P_r \times P_c = P$, costruita con `MPI_Cart_create`. La chiamata restituisce un nuovo comunicatore nel quale ogni processo ha, oltre al proprio rank, una coppia di coordinate recuperabile con `MPI_Cart_coords`: sono le (r, c) della sezione precedente, ed è su di esse che il codice ragiona da qui in avanti.

Due parametri della chiamata meritano una nota:

- la griglia è dichiarata **non periodica**. La periodicità serve agli schemi in cui ogni processo comunica con i propri vicini e il bordo deve richiudersi sul lato opposto; qui le comunicazioni sono collettive lungo un'intera riga o colonna, e un collegamento fra l'ultimo processo e il primo non avrebbe alcun significato;
- `reorder = 1` autorizza l'implementazione a **rinumerare** i processi rispetto a `MPI_COMM_WORLD`, per far corrispondere meglio la griglia logica alla topologia fisica della macchina. È la libertà che rende utile una topologia virtuale, ma ha una conseguenza da non dimenticare: da quel momento l'unico rank valido è quello del nuovo comunicatore, e assumere che coincida con quello di partenza sarebbe un errore.

La forma della griglia può essere imposta dall'utente con i flag `-pr` e `-pc`. In assenza di indicazione, `grid_default_shape` sceglie la fattorizzazione più vicina alla forma quadrata, prendendo come P_r il più grande divisore di P non superiore a $\sqrt{P}$; ne segue $P_r \leq P_c$, e nel caso limite di P primo la griglia degenera in $1 \times P$. La ragione per cui il quadrato è un buon default, e i casi in cui non lo è, sono in sezione 3.8.

3.3.1 I due sotto-comunicatori

La griglia da sola non basta: le collettive vanno eseguite lungo una riga o lungo una colonna, non sull'insieme di tutti i processi, e una collettiva agisce su un comunicatore intero. Servono quindi comunicatori più piccoli, uno per ogni riga e uno per ogni colonna della griglia.

Li produce `MPI_Cart_sub`, a cui si indica quali dimensioni lasciar variare e quali tenere fisse. Fissando la coordinata di riga e lasciando variare quella di colonna si ottiene `row_comm`, che raccoglie i processi di una stessa *riga*; fissando la colonna si ottiene `col_comm`, che raccoglie quelli di una stessa *colonna*. Ogni processo appartiene esattamente a un `row_comm` e a un `col_comm`, e sono i due assi lungo cui viaggiano le uniche due collettive del cammino cronometrato.

3.3.2 Chi fa da root, e perché la scelta è verificata

Entrambe le collettive hanno bisogno di un root, e in entrambi i casi il processo giusto è determinato dal problema, non arbitrario: il broadcast di X deve partire dal processo di riga 0, che è l'unico a possedere inizialmente quella fetta, e la reduce di Y deve arrivare al processo di colonna 0, che è il proprietario designato di quella porzione del risultato.

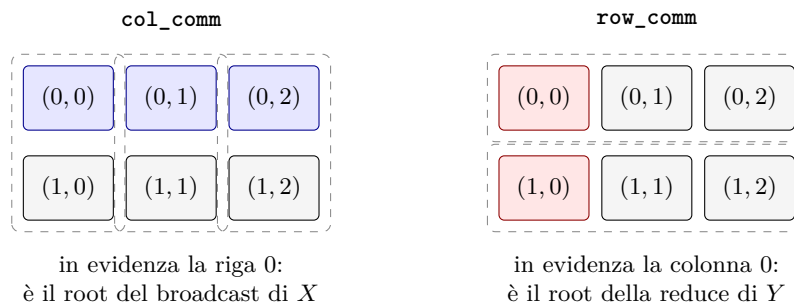


Figura 3.2: La stessa griglia, raggruppata nei due modi. Fissando la coordinata di riga si ottengono i comunicatori `row_comm`, fissando quella di colonna i comunicatori `col_comm`. Ogni processo appartiene a esattamente uno di ciascun tipo, e i due si intersecano soltanto in lui.

Indicare il root a una collettiva significa però indicarne il *rank nel sotto-comunicatore*, non le coordinate nella griglia. Il codice sfrutta allora una proprietà di `MPI_Cart_sub`: in un sotto-comunicatore con una sola dimensione residua i rank risultano ordinati per coordinata crescente. Ne segue che il rank 0 di `col_comm` è il processo di riga 0, e il rank 0 di `row_comm` è il processo di colonna 0: in entrambi i casi il root cercato, e le due collettive possono usare semplicemente 0.

La proprietà viene comunque **verificata** in `grid_create`, confrontando il rank in ciascun sotto-comunicatore con la coordinata cartesiana corrispondente. Il motivo è la libertà concessa da `reorder = 1`: se l'ipotesi cadesse, le collettive continuerebbero a funzionare regolarmente, ma con il root sbagliato, e il programma produrrebbe risultati errati senza segnalare nulla.

3.4 Partizionamento degli indici

Decidere la forma della griglia non basta: bisogna ancora stabilire, per ciascuna delle due dimensioni, quali indici globali finiscano su quale parte. La regola è una sola e viene applicata due volte — alle righe di A fra le P_r righe della griglia, alle colonne fra le P_c colonne — ed è una partizione **a blocchi**: ogni parte riceve un intervallo contiguo di indici, e il resto della divisione viene distribuito un elemento per volta sulle prime parti.

Il blocco contiguo non è una scelta di comodo. L'alternativa naturale sarebbe una distribuzione ciclica, che assegna gli indici a turno; ma qui il lavoro per elemento è uniforme, quindi il ciclico non bilancerebbe nulla che il blocco già non bilanci, e in cambio distruggerebbe la contiguità — cioè proprio la proprietà da cui dipendono sia il datatype derivato della distribuzione (sezione 3.6) sia il passo unitario degli accessi dello schema A (sezione 2.3).

3.4.1 Le funzioni

Posti $q = \lfloor n/p \rfloor$ e $r = n \bmod p$, le prime r parti hanno $q + 1$ elementi e le restanti q . Da qui le prime due funzioni, che dicono *quanti* elementi tocchino alla parte i e *da dove* cominci:

$$\text{size}(n, p, i) = q + \begin{cases} 1 & i < r \\ 0 & \text{altrimenti} \end{cases} \quad \text{start}(n, p, i) = i q + \min(i, r). \quad (3.1)$$

Il $\min(i, r)$ nell'offset conta quante parti *lunghe* precedono la parte i : finché $i < r$ sono tutte lunghe e se ne accumula una per ciascuna, dopo il loro numero resta fermo a r . La funzione è definita anche per $i = p$, dove vale n : una sentinella che rende l'ultima parte un caso come gli altri.

Lo sbilanciamento massimo è di un elemento per dimensione: fra due blocchi locali qualsiasi c'è al più una riga e una colonna di differenza.

3.5 Layout dei blocchi locali

Applicando la partizione alle due dimensioni si ottiene la forma dei tre blocchi che ogni processo possiede:

$$A_{\text{loc}} : m_{\text{loc}} \times n_{\text{loc}}, \quad X_{\text{loc}} : n_{\text{loc}} \times k, \quad Y_{\text{loc}} : m_{\text{loc}} \times k, \quad (3.2)$$

con $m_{\text{loc}} = \text{size}(M, P_r, \text{my_row})$ e $n_{\text{loc}} = \text{size}(N, P_c, \text{my_col})$.

3.5.1 Perché i conteggi delle collettive tornano

Nelle due formule qui sopra c'è una proprietà che vale la pena rendere esplicita, perché è la condizione di consistenza delle due collettive: m_{loc} dipende **solo** da **my_row** e n_{loc} **solo** da **my_col**.

Ne segue che tutti i processi di una stessa riga della griglia concordano su m_{loc} , e tutti quelli di una stessa colonna concordano su n_{loc} . Sono esattamente i due gruppi su cui agiscono le collettive: il broadcast lungo **col_comm** usa un conteggio di $n_{\text{loc}} \cdot k$ elementi, e i suoi partecipanti condividono la colonna, dunque n_{loc} ; la reduce lungo **row_comm** usa $m_{\text{loc}} \cdot k$, e i suoi partecipanti condividono la riga, dunque m_{loc} . Nessuno dei due conteggi va negoziato o comunicato: viene dalla stessa formula, valutata su coordinate che i partecipanti hanno in comune.

3.5.2 Chi contiene che cosa, e quando

Il buffer X_{loc} esiste su *tutti* i processi — il broadcast deve avere dove scrivere — ma prima del broadcast contiene dati significativi soltanto sui processi della riga 0. Dopo il prodotto locale ogni processo possiede un contributo *parziale* a Y , che riguarda le proprie righe ma soltanto le proprie colonne di A ; la `MPI_Reduce` lungo `row_comm` li somma e produce il blocco definitivo sui processi della colonna 0.

3.5.3 Perché il multivettore non viene mai tagliato lungo k

Le k colonne restano indivise su tutti i processi. Tre ragioni, in ordine di peso:

- **costringerebbe a replicare A .** Per calcolare un qualunque sottoinsieme di colonne di Y serve comunque *tutta* A : i processi di una stessa partizione lungo k vorrebbero perciò lo stesso identico blocco di A . Si duplicherebbe il dato dominante (`??`), che è l'opposto di ciò per cui la decomposizione esiste;
- **ridurrebbe il riuso di A in registro.** Nello schema A il fattore di riuso è pari al numero di colonne che il processo possiede localmente (sezione 2.3): frazionare k lo frazionerebbe nella stessa misura, abbassando l'intensità aritmetica del nucleo;
- **produrrebbe messaggi troppo piccoli.** k è piccolo per definizione del problema, e frammentarlo ulteriormente darebbe comunicazioni dominate dalla latenza invece che dalla banda.

Invariante critico

Il numero di righe locali di X deve venire dalla *stessa* partizione con cui si tagliano le colonne di A , e il numero di righe locali di Y dalla stessa con cui se ne tagliano le righe. Non è un controllo da eseguire, è una proprietà garantita *per costruzione*: esistono soltanto m_{loc} e n_{loc} , calcolati una volta sola in `layout_init`, e non due coppie di valori da tenere allineate. La struttura `layout_t` è il punto unico di verità sulla forma dei blocchi, e finché resta tale l'invariante non può essere violato senza modificarla.

3.6 Distribuzione iniziale dei dati

Le modalità di inizializzazione di A e X sono indipendenti (`-a-mode`, `-x-mode`), e tutte e quattro le combinazioni sono valide. Le due modalità di ciascuna matrice producono gli stessi elementi globali a parità di seed, per la proprietà del generatore discussa in sezione 2.5.

-a-mode global. Il grid rank 0 materializza A completa e la distribuisce. È il caso interessante dal punto di vista MPI: i sottoblocchi 2D **non sono contigui** nella matrice row-major globale. Il mittente descrive il blocco da inviare con un `MPI_Type_vector` di m_{loc} blocchi da n_{loc} elementi e stride N . Il destinatario ne descrive uno con stride `lda`, che può essere diverso: è il caso generale - due layout differenti sui due lati della stessa comunicazione - e non richiede nessun buffer intermedio né nessuna copia manuale. Il rank 0 fa eccezione per il proprio blocco, che copia direttamente senza passare da MPI.

-a-mode local. Ogni processo genera direttamente il proprio blocco 2D di A dagli indici globali. In questo modo non è necessario materializzare una copia globale aggiuntiva della matrice su un singolo processo.

-x-mode global Il grid rank 0 materializza la X compatta $N \times k$ e un `MPI_Scatterv` sul `row_comm` della riga 0 ne distribuisce i blocchi di righe: entrambi i lati sono contigui, perché $ldx = k$ ovunque, quindi non serve nessun datatype derivato.

-x-mode local I processi della riga 0 generano direttamente le proprie righe di X . Il successivo broadcast lungo `col_comm` è identico alla modalità `global`.

Generazione e distribuzione iniziale fanno parte del **preprocessing**, perciò sono escluse dai tempi ufficiali. La modalità `global` rappresenta uno scenario in cui l'input è inizialmente centralizzato.

3.7 L'algoritmo distribuito

Un'invocazione del prodotto esegue tre passi:

```
MPI_Bcast(X_loc, n_loc*k, SCALAR_MPI_TYPE, 0, grid->col_comm);

local_gemm(local_gemm_context, X_loc, layout->ldx, Y_loc_part, layout->ldy);

MPI_Reduce(Y_loc_part, Y_row_col0, m_loc*k, SCALAR_MPI_TYPE,
           MPI_SUM, 0, grid->row_comm);
```

Listato 3.1: Il corpo di `mpi_matmul`, senza i cronometri.

1. **Broadcast di X lungo `col_comm`:** tutti i processi di una colonna della griglia lavorano sulle stesse colonne di A , quindi servono loro le stesse righe di X . Il root è il processo di riga 0, l'unico che possiede inizialmente quella fetta, ne è il *responsabile*.
2. **Prodotto locale:** il risultato è *parziale*: il processo in posizione (r, c) calcola il contributo delle sole colonne $[col_0, col_0 + n_{loc})$ di A alle righe $[row_0, row_0 + m_{loc})$ di Y (dove row_0 e col_0 sono gli offset globali del blocco locale).
3. **Reduce lungo `row_comm`:** i P_c contributi parziali della stessa banda di righe vivono sui processi di una stessa riga della griglia: sommandoli tra loro si ottiene Y . Il count è $m_{loc} \cdot k$ (e non 1) dato che la riduzione agisce lungo la dimensione dei processi e preserva la posizione, producendo dunque $m_{loc} \cdot k$ somme indipendenti. Il root è il processo di colonna 0, che è già il *responsabile* designato di quella porzione di Y : nessuna ridistribuzione finale è necessaria.

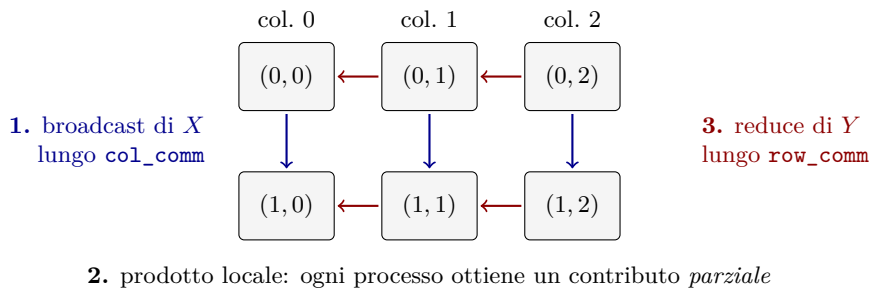


Figura 3.3: Le due collettive di una singola invocazione. Il broadcast fa scendere la fetta di X dal processo di riga 0 a tutti i processi della sua colonna, che ne hanno bisogno perché condividono le stesse colonne di A . Dopo il prodotto locale ogni processo possiede un contributo *parziale* a Y : la reduce somma i P_c parziali di una stessa banda di righe e deposita il risultato sul processo di colonna 0, che ne è il proprietario designato.

3.8 Modello di costo della comunicazione

Il costo di un messaggio di n byte fra due processi si modella, seguendo le dispense del corso, come

$$T(n) = \lambda + \frac{n}{B}, \quad (3.3)$$

dove λ è la *latenza*, un costo fisso che si paga per ogni messaggio indipendentemente dalla sua taglia, e B è la *banda*. In letteratura lo stesso modello compare spesso nella forma $\alpha + \beta n$: il dizionario è $\alpha \equiv \lambda$ e $\beta \equiv 1/B$.

Le due collettive non sono però messaggi singoli. Il costo logaritmico che segue è quello del *recursive doubling*, l'algoritmo di broadcast visto a lezione, che al posto dei $p-1$ invii sequenziali della versione ingenua costa $T(p) = \lceil \log_2 p \rceil$ tempi di messaggio. Il logaritmo si ottiene grazie a un'ipotesi precisa sulla rete — che essa sostenga più messaggi contemporaneamente, purché le coppie di nodi coinvolte non si sovrappongano — ed è quell'ipotesi, più che la struttura ad albero, a rendere possibile il guadagno.

Detti s i byte per scalare, il costo di comunicazione per invocazione è quindi

$$T_{\text{comm}} \approx \underbrace{\lceil \log_2 P_r \rceil \left(\lambda + \frac{s k N / P_c}{B} \right)}_{\text{broadcast di } X} + \underbrace{\lceil \log_2 P_c \rceil \left(\lambda + \frac{s k M / P_r}{B} + \gamma k \frac{M}{P_r} \right)}_{\text{reduce di } Y}, \quad (3.4)$$

dove γ è il costo per elemento della somma eseguita dalla reduce. Il termine in γ non compare nel modello del corso, che descrive il costo del solo trasferimento: lo aggiungiamo perché la reduce, a differenza del broadcast, a ogni livello dell'albero esegue anche $m_{\text{loc}} \cdot k$ addizioni.

Il modello va letto per quello che è: i fattori logaritmici valgono se l'implementazione usa il recursive doubling, ma le librerie MPI cambiano algoritmo al crescere del messaggio e per messaggi grandi passano a schemi a pipeline o scatter/allgather, nei quali il termine di banda perde il fattore logaritmico e resta proporzionale al solo volume trasferito.

Il modello va letto per quello che è: i fattori logaritmici valgono se l'implementazione usa il recursive doubling, ma — come le dispense stesse osservano — una buona libreria MPI commuta internamente fra algoritmi diversi a seconda dell'operazione, della rete e soprattutto della quantità di dati. Per messaggi grandi si passa a schemi a pipeline o scatter/allgather, nei quali ogni byte attraversa la rete una volta sola invece di $\log_2 p$ volte: il termine di banda perde il fattore logaritmico e resta proporzionale al solo volume, al prezzo di un numero maggiore di messaggi e quindi di più latenza.

Alle taglie della campagna i messaggi cadono in quel regime, quindi il $\log_2$ va inteso come limite superiore sui termini di latenza più che come andamento atteso della banda. Questo non intacca però la conclusione che segue: il *volume*, cioè l'unica quantità da cui dipende la scelta della forma della griglia, è lo stesso per qualunque algoritmo collettivo, perché è quanto ciascun processo deve comunque ricevere o inviare.

Il termine che decide la forma ottima della griglia è il volume:

$$V(P_r, P_c) = k \left(\frac{N}{P_c} + \frac{M}{P_r} \right), \quad P_r P_c = P. \quad (3.5)$$

Minimizzando con il vincolo $P_r P_c = P$ si ottiene

$$P_r^* = \sqrt{P \frac{M}{N}}, \quad P_c^* = \sqrt{P \frac{N}{M}}, \quad V^* = \frac{2k\sqrt{MN}}{\sqrt{P}}. \quad (3.6)$$

Due letture, entrambe verificate sperimentalmente in capitolo 9:

- **Perché 2D e non 1D.** Le due decomposizioni monodimensionali sono i casi degeneri $P_r = 1$ (volume kM , nessun broadcast ma reduce su tutti i P processi) e $P_c = 1$ (volume kN , nessuna reduce ma broadcast su tutti i P). In entrambi il volume *non decresce* con P , mentre la griglia 2D lo fa scendere come $1/\sqrt{P}$. È il motivo per cui la traccia richiede una griglia bidimensionale, ed è ciò che la campagna C6 misura direttamente confrontando tutte le fattorizzazioni di P , forme degeneri comprese.
- **Perché la forma ottima dipende da $M:N$.** Per $M = N$ la griglia migliore è quella quadrata, cioè proprio quella che `grid_default_shape` sceglie da sé — ed è anche quella che lo standard MPI sceglierebbe da sé, visto che `MPI_Dims_create` costruisce la griglia più vicina possibile alla forma quadrata. La eq. (3.6) dice però che quel default è ottimo *soltanto* per $M = N$; per $M = 3N$ conviene invece $P_r > P_c$. La campagna C1 varia il rapporto $M:N$ a lavoro costante e rende visibile questo effetto separandolo da quello della taglia.

Un'osservazione che va tenuta presente leggendo il weak scaling (campagna C5): su una griglia 2D con blocco locale costante, M segue P_r e N segue P_c , quindi cambiare P cambia *anche* il rapporto $M:N$. È inerente alla decomposizione, ed è la ragione per cui l'effetto della forma viene misurato separatamente in C1 e C6.

Capitolo 4

Il nucleo di CPU

Il nucleo di CPU non è la parte più veloce del progetto, ma è quella di cui tutto il resto ha bisogno per essere giudicato: è il denominatore di ogni speedup, ed è l'unica implementazione che gira ovunque, anche su una macchina senza `nvcc`. Merita quindi lo stesso lavoro dei backend CUDA, e per le stesse ragioni.

Il vincolo che se ne è dato il progetto è che sia C11 portabile, senza `intrinsic`s e senza direttive di vettorizzazione. Non è una rinuncia: è una scelta che sposta il problema. Senza `intrinsic`s l'unica leva sulla vettorizzazione è la *forma* del codice, che deve essere abbastanza semplice e abbastanza esplicita da lasciare al compilatore una decisione facile. Buona parte di questo capitolo riguarda proprio come si scrive un ciclo perché un compilatore lo vettorizzi da sé.

4.1 Tre backend, una sola idea

Tutti e tre i backend di CPU implementano lo schema A di sezione 2.3: cambia come il lavoro viene organizzato attorno a quell'ordine dei cicli, non l'ordine. Si scelgono a tempo di compilazione con la variabile `KERNEL` del `Makefile`, espongono la stessa interfaccia (??) e i binari coesistono, così un confronto è una sola campagna sugli stessi dati.

- `scheme_a` — l'implementazione di riferimento, quella che corrisponde alla consegna;
- `scheme_a_jblock` — la stessa, con X mattonellata in cache;
- `scheme_a_jblock_rb` — come sopra, più il *register blocking* sulle righe di A .

I tre non sono tre tentativi alternativi ma una **catena**: ciascuno rimuove il collo di bottiglia che il precedente lascia scoperto. Il capitolo li presenta in quest'ordine, e ogni sezione si apre con il limite che motiva il backend successivo.

4.2 Il backend di riferimento

4.2.1 Perché k deve essere noto al compilatore

Nello schema A i k accumulatori sono variabili locali del ciclo su j , e tutto il vantaggio dello schema dipende dal fatto che vivano nei **registri**: è lì che il valore $A[i][j]$, letto una volta sola, viene riusato k volte.

Perché il compilatore possa tenerceli deve però sapere *quanti* sono mentre compila. Il motivo è concreto: i registri non sono indirizzabili a runtime. Se il numero di accumulatori è noto solo quando il programma parte, `acc` diventa per forza un array in memoria e ogni aggiornamento

`acc[c] += ...` un accesso a quella memoria — esattamente il traffico che lo schema A serviva a evitare.

Il codice risolve generando **cinque funzioni specializzate**, una per ciascuno dei k richiasti dalla traccia, a partire da una sola definizione via macro:

```
#define COLS_8(M) COLS_6(M) M(6) M(7)
#define DECLARE_ACC(c) scalar_t acc##c = (scalar_t)0;
#define UPDATE_ACC(c) acc##c += a * xrow[c];
#define STORE_ACC(c) yrow[c] = acc##c;

/* per ogni riga i: */
COLS(DECLARE_ACC)
for (j = 0; j < n_loc; j++) {
    const scalar_t a = arow[j];          /* letto UNA volta */
    const scalar_t *restrict xrow = X_loc + (size_t)j * ldx;
    COLS(UPDATE_ACC)                    /* k aggiornamenti espliciti */
}
COLS(STORE_ACC)
```

Listato 4.1: Generazione delle specializzazioni (estratto di `src/kernel/scheme_a.c`).

Vale la pena leggere che cosa producono queste tre righe. `COLS_8` è un elenco di indici; applicarlo a `DECLARE_ACC` genera otto dichiarazioni di variabili scalari *distinte* (`acc0`, `acc1`, ...), non un array; applicarlo a `UPDATE_ACC` genera otto aggiornamenti scritti per esteso, senza alcun ciclo interno con estremo runtime. Il percorso caldo che il compilatore vede non contiene quindi nessuna indicizzazione su c : contiene k istruzioni indipendenti su k variabili indipendenti, che è la forma in cui l’allocazione in registro e la vettorizzazione sono decisioni facili.

4.2.2 Il fallback generico, e perché la traccia lo richiede

La traccia chiede però che il codice funzioni per k *generico*, e cinque specializzazioni non possono coprire tutti i valori possibili. Per ogni altro k interviene un **fallback generico**, che non tenta di tenere k accumulatori — sarebbero troppi, e finirebbero in memoria — ma lavora a gruppi di al più $\text{KB} = 32$ colonne per volta. Ogni gruppo richiede una passata completa su A :

$$\text{passaggi su } A = \left\lceil \frac{k}{\text{KB}} \right\rceil. \quad (4.1)$$

Per $k \leq \text{KB}$ il ciclo esterno sui gruppi è *degenere*: esiste nel codice ma gira una volta sola, A viene letta una volta sola e il fallback si differenzia dalle specializzazioni soltanto per l’array di accumulatori invece delle variabili distinte. Oltre quella soglia i passaggi diventano due, e con essi raddoppia il traffico sia su A sia su X .

La conseguenza va tenuta presente leggendo i risultati: le righe dei k oltre la soglia e quelle dei k entro la soglia **non sono direttamente confrontabili**, perché non stanno facendo la stessa quantità di lavoro di memoria. È l’andamento a dente di sega già anticipato in sezione 2.3.

4.2.3 Il microbenchmark che isola la specializzazione

Che le specializzazioni servano davvero è un’ipotesi, e va misurata. Compilando con `FORCE_GENERIC K=1` le cinque funzioni non vengono nemmeno generate e ogni k passa dal fallback: le due build differiscono per una sola variabile, coesistono come binari distinti e `kernel_name()` le distingue nel CSV (`scheme_a` contro `scheme_a_generic`). Il confronto è quindi un A/B sullo stesso problema e nella stessa sessione di misura, non due esecuzioni accostate a posteriori.

La campagna include un **controllo di consistenza** che vale la pena dichiarare, perché è ciò che rende il risultato credibile: sui k oltre la soglia entrambe le build passano dal fallback, quindi lì le due righe devono coincidere entro il rumore. Se non coincidessero, la differenza misurata sugli altri k non sarebbe specializzazione ma rumore, e la campagna andrebbe rifatta.

4.3 Perché $KB = 32$

Il valore copre tutti i k del collaudo, ma questa da sola non è una ragione sufficiente: sarebbe una costante scelta per far tornare i casi che interessano. La ragione vera è un budget di registri.

Con KB accumulatori il compilatore deve tenere KB valori vivi attraverso l'intero ciclo su j . Vettorizzati, occupano KB/W registri vettoriali, dove W è il numero di scalari che entrano in un registro — quattro in doppia precisione su AVX2, otto su AVX-512 — a fronte dei 16 o 32 registri architetturali disponibili. Al crescere di KB il riuso di A migliora, perché i passaggi della eq. (4.1) diminuiscono, ma cresce la pressione sui registri fino allo *spill*, cioè fino al punto in cui il compilatore è costretto a parcheggiare in memoria gli accumulatori che non stanno più nei registri — e a quel punto si perde più di quanto si guadagni.

Il valore 32 è il compromesso che tiene i cinque k richiesti in un passaggio solo restando sotto la soglia di spill sulle architetture di interesse. Che lo sia davvero anche sui k grandi è una previsione, non un dato, ed è uno degli oggetti della campagna C3 (sezione 9.2).

4.4 Ciò che il compilatore vede

La build usa `-O3` con `-march=native` (oppure `-mcpu=native` su aarch64: il Makefile prova quale delle due il compilatore accetta), più `-Wall -Wextra -Wpedantic`. Nessun `#pragma`, nessun `__restrict`.

Perché questo basti, il codice deve rispondere da sé alle tre domande che un compilatore si pone prima di vettorizzare un ciclo.

- «**Questi puntatori possono riferirsi alla stessa memoria?**» No: A , X e Y sono dichiarati `restrict`. La garanzia va però *ripristinata* dentro `local_gemm`, con copie locali dei puntatori: il contesto opaco tiene il puntatore ad A come membro di una struttura, e senza quella copia i kernel specializzati perderebbero l'informazione di non aliasing che avevano quando A era un parametro. Se l'aliasing fosse possibile, ogni scrittura su Y potrebbe invalidare A e il ciclo andrebbe eseguito in ordine.
- «**Le iterazioni dipendono l'una dall'altra?**» No: i k aggiornamenti `acc[c] += a * X[j][c]` riguardano accumulatori distinti e sono indipendenti fra loro. Essendo già srotolati dalle macro, si presentano come catene di moltiplicazione-accumulo indipendenti, che è la forma che una unità FMA può eseguire in parallelo.
- «**Gli accessi sono regolari?**» Sì, e a passo unitario: come osservato in sezione 2.3, sia la riga di A sia la riga di X sono contigue in memoria.

Da completare con i dati della campagna

Verificare sul server l'effettiva vettorizzazione del percorso caldo, per esempio con `gcc -fopt-info-vec` oppure ispezionando l'assembly di `kernel_k8` e `kernel_k32`. Il punto da documentare è se e a quale k compaiano spill degli accumulatori.

4.5 Il primo limite: la riletture di X

Lo schema A legge A una volta sola, ed è il suo pregio. Ma per ogni riga i di A percorre **tutta** X , e questo è il suo prezzo: il traffico in lettura su X vale

$$\text{elementi di } X \text{ letti} = m_{\text{loc}} \cdot n_{\text{loc}} \cdot k, \quad (4.2)$$

cioè k volte il traffico su A .

Finché X resta in una cache vicina al calcolo questo traffico non raggiunge la memoria centrale e semplicemente non si vede. Quando X non ci sta più, viene ristreamata da un livello più lontano una volta per ogni riga di A , e il tempo per elemento smette di essere governato dal calcolo per essere governato dalla banda di quel livello.

La soglia è esplicita e dipende da entrambi i parametri del problema: il blocco locale X_{loc} occupa $n_{\text{loc}} \cdot k \cdot s$ byte. A parità di matrice, quindi, è **k che decide da che parte della soglia ci si trova**, ed è per questo che l'effetto si manifesta come un calo dei GFLOPS al crescere di k — cioè nella direzione opposta a quella che l'intensità aritmetica della eq. (2.3) lascerebbe prevedere. È la chiave di lettura dei risultati di sezione 9.2, ed è il limite che il backend successivo rimuove.

4.6 Secondo backend: X mattonellata in cache

L'idea di `scheme_a_jblock` è di non percorrere più tutta X per ogni riga di A , ma di prenderne un **tile** — un blocco di righe consecutive — e di applicarlo a tutte le righe di A prima di passare al successivo:

```
for j0 (tile di X di BJ righe):
  for i (TUTTE le righe di A):
    acc = (primo tile) ? 0 : Y[i][*]
    for j nel tile: a = A[i][j]; acc[c] += a * X[j][c]
    Y[i][*] = acc
```

Listato 4.2: Struttura di `src/kernel/scheme_a_jblock.c`.

Il tile viene caricato dalla memoria una volta e riusato m_{loc} volte restando in cache. A continua a essere letta *una volta sola*: ogni sua riga viene semplicemente consumata a segmenti, uno per tile, invece che tutta di seguito.

Il prezzo. Y non può più vivere nei registri per tutta la durata di una riga, perché ci si torna sopra a ogni tile: va riletta e riscritta ogni volta. Il traffico aggiuntivo vale $2k$ scalari per riga e per tile, cioè un rapporto $2k/\text{BJ}$ rispetto al traffico su A . Il tile va quindi scelto *grande* abbastanza da rendere questo termine trascurabile e *piccolo* abbastanza da stare in cache: le due richieste tirano in direzioni opposte, e il punto di equilibrio non è deducibile a priori.

Per questo la dimensione del tile è un parametro di build, `JBLOCK_BYTES`, e non una costante scelta una volta per tutte: il codice ne ricava il numero di righe per tile dividendo per $k \cdot s$, il default è un valore che sta nella cache di secondo livello di qualunque core x86 recente, e quale sia il valore migliore è oggetto di misura.

Un principio già visto. È esattamente la stessa mossa del backend CUDA con tile in shared memory (??): portare un blocco di X in una memoria vicina al calcolo e riusarlo per molte righe prima di passare al blocco successivo. Cambia il nome della memoria vicina — lì la shared memory, qui la cache — non il ragionamento.

4.7 Terzo backend: register blocking sulle righe

Tolto il limite di banda, ne resta scoperto un altro, che si manifesta ai *k piccoli*: in tutti i backend visti finora ogni `acc[c]` è una catena *seriale* di moltiplicazioni-accumulo lungo *j* — ogni aggiornamento deve attendere il precedente — e l'unico parallelismo a livello di istruzione disponibile è quello fra le *k* colonne. Poiché una unità FMA ha una latenza di diversi cicli e la CPU ne ha più di una, per tenerle occupate servono più catene indipendenti di quante *k* piccolo ne offra: il processore aspetta, e i GFLOPS crescono con *k* invece di essere costanti.

`scheme_a_jblock_rb` risolve lavorando su RB righe di *A* per volta:

```
for j0 (tile di X, come in scheme_a_jblock):
  for i a passi di RB:
    acc[r][c] = (primo tile) ? 0 : Y[i+r][c]
    for j nel tile:
      x[c] = X[j][c]                                /* caricato UNA volta */
      for r: a = A[i+r][j]; acc[r][c] += a * x[c]    /* riusato RB volte */
    Y[i+r][c] = acc[r][c]
```

Listato 4.3: Struttura di `src/kernel/scheme_a_jblock_rb.c`.

Le righe di *A* sono indipendenti per costruzione — nessun elemento di *Y* dipende da un'altra riga — quindi il numero di catene indipendenti passa da $\lceil k/W \rceil$ a $RB \cdot \lceil k/W \rceil$. In più, ogni valore di *X* caricato viene riusato RB volte direttamente dai registri, quindi anche il traffico su *X* — già in cache grazie al tile — si riduce dello stesso fattore.

Chi sceglie RB. Anche qui il limite sono i registri: $RB \cdot \lceil k/W \rceil$ accumulatori più RB valori di *A* devono starci tutti, altrimenti si torna allo spill e si perde più di quanto si guadagna. Il codice sceglie perciò RB *a tempo di compilazione e per ogni k*, dal numero di registri vettoriali e dalla loro ampiezza dell'architettura di destinazione (`VEC_NREGS` e `VEC_W`, fissati dalle macro del compilatore), prendendo il più grande fra i valori ammessi che soddisfi il vincolo; la variabile di build `RB_ROWS` permette di forzarlo per misurarne l'effetto. Le righe che avanzano quando m_{loc} non è multiplo di RB sono chiuse dal kernel a una riga, che è il caso $RB = 1$ dello stesso schema.

I tre backend rispondono quindi a tre limiti diversi e successivi: il traffico su *A*, risolto dallo schema A stesso; la banda verso *X*, risolta dal tile in cache; il parallelismo di istruzione, risolto dal blocco di righe. Quanto ciascuno dei tre passi valga davvero sulla macchina di misura è ciò che la campagna riportata in capitolo 9 quantifica.

Capitolo 5

I backend CUDA

Il progetto contiene quattro kernel CUDA e un riferimento esterno. Non sono alternative in competizione: sono una **sequenza**, in cui ogni versione cambia una sola cosa rispetto alla precedente, così che ogni differenza di tempo misurata abbia una causa identificabile.

Tabella 5.1: I backend GPU e la variabile che ciascuno cambia.

Backend	Mappatura	Che cosa cambia rispetto al precedente
<code>cuda_naive</code>	un thread per elemento di Y	baseline: riuso di A in registro perso
<code>cuda_warp</code>	un warp per riga di Y	recupera riuso e coalescenza; k accumulatori per lane
<code>cuda_warp_smem</code>	come sopra	la fetta di X passa dalla shared memory
<code>cuda_warp_tiled</code>	un warp per tile di colonne di una riga	meno accumulatori per lane, più letture di A
<code>cublas</code>	<code>cublas<t>gemm</code>	riferimento esterno, non una proposta

5.1 Struttura comune

Tutti condividono il ciclo di vita di sezione 2.6.1. La ripartizione fra preprocessing e cammino cronometrato è la stessa per tutti, ed è la condizione perché i loro tempi siano confrontabili:

Fase	Dove	Cronometro
creazione del contesto CUDA	<code>local_gemm_create</code>	no: $t_{\text{setup}}^{\text{init}}$
tutti i <code>cudaMalloc</code> in VRAM	<code>local_gemm_create</code>	no: $t_{\text{setup}}^{\text{alloc}}$
H2D di A , una volta sola	<code>local_gemm_create</code>	no: $t_{\text{setup}}^{\text{H2D } A}$
H2D di X , kernel, D2H di Y	<code>local_gemm</code>	sì: t_{local}
solo il kernel (<code>cudaEvent</code>)	dentro <code>local_gemm</code>	sì: t_{kernel}
solo l'H2D di X (<code>cudaEvent</code>)	dentro <code>local_gemm</code>	a parte: $t_{\text{H2D } X}$
solo la D2H di Y (<code>cudaEvent</code>)	dentro <code>local_gemm</code>	a parte: $t_{\text{D2H } Y}$

Tabella 5.2: Che cosa entra in quale cronometro, uguale per tutti i backend GPU.

Perché i `cudaEvent` e non l'orologio dell'host. Un lancio di kernel è asincrono: l'orologio dell'host misurerebbe il tempo di *accodamento*, non quello di esecuzione. Solo il backend può misurare t_{kernel} , e solo con event registrati sullo stream. Per questo l'interfaccia espone `local_gemm_last_compute_seconds`: è il canale con cui un backend che sa distinguere il kernel dal resto pubblica il dato, e i backend di CPU restituiscono un valore negativo per dire “non applicabile”.

Lo stesso vale, e con un argomento ancora più stringente, per la D2H di Y : quella copia parte solo quando il kernel ha finito, quindi un cronometro sull’host misurerebbe “attesa del kernel + copia” e attribuirebbe al PCIe del tempo di calcolo. Gli event non sono qui una raffinatezza, sono la condizione perché il numero significhi ciò che dice.

La fase locale è scomposta per intero. La differenza $t_{\text{local}} - t_{\text{kernel}}$ resta nel CSV come *transfer/runtime overhead*, ma non è più l’unica informazione disponibile, e da sola non era *discutibile*: metteva nello stesso numero il PCIe e il runtime dell’host, che hanno cause diverse e scalano in modo diverso — il primo con $n_{\text{loc}}k$ e $m_{\text{loc}}k$, il secondo per niente. I due trasferimenti hanno quindi un canale ciascuno, e l’overhead del runtime è ciò che **avanza**:

$$t_{\text{local}} = t_{\text{H2D } X} + t_{\text{kernel}} + t_{\text{D2H } Y} + t_{\text{launch}}. \quad (5.1)$$

Solo un backend che misura tutte e tre le voci nominate può produrre t_{launch} ; altrove resta la sentinella negativa. Nessuna delle voci di destra oltre t_{kernel} entra nella metrica ufficiale: sono esattamente i tempi che la traccia esclude da T e consente di misurare e discutere a parte (??).

Dai tempi alla banda. I tre trasferimenti espongono anche i *byte* trasferiti (`local_gemm_bytes_h2d_A` e le due per invocazione), così che il driver possa pubblicare `bw_h2d_A_gbs`, `bw_h2d_X_gbs` e `bw_d2h_Y_gbs` e confrontarle con il picco del bus. Senza i byte, un tempo di copia non si può giudicare: dice che la copia costa, non se stia andando alla velocità giusta.

Anche il preprocessing è scomposto. Il totale t_{setup} dice che la preparazione costa, non *perché*. Su CUDA le tre voci che lo compongono hanno cause e andamenti diversi, e vanno separate per essere discusse:

Voce	Che cos’è e come scala
$t_{\text{setup}}^{\text{init}}$	creazione del contesto CUDA: costo fisso di qualche centinaio di millisecondi, indipendente dalla taglia
$t_{\text{setup}}^{\text{alloc}}$	i <code>cudaMalloc</code> di A , X e Y in VRAM: cresce con la taglia, ma non è un trasferimento
$t_{\text{setup}}^{\text{H2D } A}$	la copia H2D di A : l’unica delle tre che sia PCIe e l’unica che scali con MN

Tabella 5.3: Scomposizione del preprocessing del backend GPU.

L’ultima è il numero più interessante da riportare, perché è esattamente il trasferimento che la scelta di separare `create` da `local_gemm` (sezione 2.6.1) ha tolto dal cammino misurato: quantificarlo è il modo onesto di dire quanto valga quella scelta. Le tre voci sommate sono $\leq$ del totale — restano fuori i controlli sugli argomenti e la creazione degli event — e valgono 0 sui backend di CPU, dove anche il totale è dell’ordine del microsecondo.

Capienza della VRAM. Non viene stimata prima, si verifica tentando. La catena di `cudaMalloc` si ferma alla prima allocazione che non entra, e solo allora `cudaMemGetInfo` viene interrogata per comporre un messaggio che dice quanta memoria serviva e quanta ce n’era. Un controllo preventivo confronterebbe una fotografia già obsoleta — sul server più rank MPI condividono l’unica GPU — e andrebbe corretto con un margine arbitrario. Un blocco locale di $40\,000 \times 40\,000$ in doppia precisione richiede 12,8 GiB contro i 15,5 GiB della scheda, e il programma si ferma prima di allocare.

Un solo device. Il server di dipartimento ha una sola GPU: tutti i rank usano il device 0 e non esiste logica di selezione. La conseguenza sperimentale è importante ed è discussa in sezione 5.7.

5.2 cuda_naive: la baseline

Il thread t calcola $Y[i][c]$ con $i = t/k$ e $c = t \bmod k$, scorrendo tutta la dimensione j con un solo accumulatore. Serve a due cose, in quest'ordine: validare la pipeline (build con `nvcc`, ciclo di vita a tre fasi, trasferimenti, misura con gli event) prima che esista il kernel che conta; e dare un numero di partenza. Uno speedup senza baseline non è un risultato.

Le sue due debolezze sono volute, perché sono esattamente ciò che le versioni successive devono battere:

- **il riuso di A in registro è perso:** ogni thread tiene un solo accumulatore, quindi la stessa riga di A viene richiesta k volte, una per ciascuno dei k thread della riga. Le richieste sono simultanee e la cache ne assorbe quasi tutte, ma il lavoro c'è — ed è il ritorno allo schema B della eq. (2.3);
- **la coalescenza è parziale:** thread consecutivi hanno c consecutivo e leggono $X[j][c]$ contigui, cioè 24 B per riga di X con $k = 3$ e 256 B con $k = 32$: da meno di una transazione a due transazioni piene.

5.3 cuda_warp: un warp per riga

Il kernel assegna un warp a ogni riga di Y e distribuisce il ciclo su j fra le 32 lane: la lane l visita $j = l, l + 32, l + 64, \dots$. Il ribaltamento è completo rispetto alla baseline:

- lane consecutive leggono elementi consecutivi di A : coalescenza piena;
- ogni lane tiene *tutti* e k gli accumulatori in registro e riusa ogni valore di A caricato per aggiornarli tutti: è lo schema A, trasposto sul warp;
- i risultati parziali delle 32 lane vengono ridotti dentro il warp con `__shfl_down_sync`, senza shared memory e senza barriere; la lane 0 scrive la riga di Y .

I cinque k della traccia sono istanze template distinte, così che il numero di accumulatori sia una costante di compilazione — lo stesso motivo per cui il nucleo di CPU genera cinque funzioni. Ogni altro k attraversa un fallback runtime a tile di quattro colonne, senza limite massimo su k .

Una nota di correttezza che vale la pena isolare: la riduzione usa la maschera piena e non `__activemask()`. Da Volta in poi le lane di un warp possono divergere e riconvergere in modo indipendente, e la documentazione CUDA dice esplicitamente che il valore di `__activemask()` non implica quali lane parteciperanno alla shuffle. Per questo il kernel è scritto in modo che tutte e 32 le lane di un warp valido raggiungano le shuffle, anche nell'ultimo blocco o quando n_{loc} è minore di 32: i warp interamente fuori intervallo escono prima, quelli parziali no.

5.3.1 Un secondo layout per X : `X_LAYOUT=column`

In `cuda_warp` la lane l visita $j = l, l + 32, \dots$, quindi le letture della riga di A sono perfettamente coalescenti — lane consecutive, indirizzi consecutivi. Le letture di X , invece, non lo sono: con X row-major la lane l legge $X[j][c]$ e due lane consecutive distano `ldx` scalari. Il traffico c'è ma è sparso, e il numero di transazioni per warp cresce.

Il rimedio non richiede di toccare il kernel: basta cambiare l'*ordine in memoria* del blocco locale di X . Con `X_LAYOUT=column` il driver, in preprocessing, riscrive X_{loc} per colonne, così che

$$X[j][c] = X_{\text{loc}}[c \cdot n_{\text{loc}} + j], \quad (5.2)$$

e allo stesso passo c le 32 lane leggono n_{loc} elementi consecutivi: anche X diventa coalescente. La selezione è una costante di compilazione, quindi non c'è nessun ramo nel percorso caldo, e la conversione è già terminata prima che il backend venga chiamato.

Tre conseguenze da enunciare, perché sono il motivo per cui l'opzione è limitata e non generale:

- **vale solo per `cuda_warp`**. Il `Makefile` la accetta solo lì e gli altri backend rifiutano `column`: attivarla altrove darebbe risultati sbagliati in silenzio, che è proprio la categoria di errore che il progetto si impegna a rendere impossibile;
- **X deve essere compatta ($\text{ldx} = k$)**: in questa build `ldx` descrive la capienza $n_{\text{loc}} \cdot \text{ldx}$ del buffer, non lo stride fra colonne, che vale n_{loc} . Il padding di X non è ammesso, e il backend lo verifica;
- **la conversione ha un costo**, ed è cronometrata a parte (`t_prep_convert_X_s`). È preprocessing — avviene una volta sola, mentre X viene poi riusata per tutte le repetition — ma riportarla è ciò che rende il confronto fra i due layout una misura completa e non solo un confronto fra due tempi di kernel.

Le due build coesistono come binari distinti (`-xcol`) e `kernel_name()` riporta il suffisso (`xcol`), quindi le righe restano distinguibili nel CSV, che porta comunque anche una colonna `x_layout` esplicita.

5.4 `cuda_warp_smem`: la fetta di X in shared memory

È `cuda_warp` con **una sola variabile cambiata**: da dove arrivano gli elementi di X . Ordine dei cicli, lettura singola di A , accumulatori in registro, riduzione per shuffle e specializzazione sui cinque k restano identici. È questo che rende il confronto una misura.

Il difetto attaccato. In `cuda_warp` ogni warp percorre tutta X , e i warp sono m_{loc} : il traffico in lettura su X vale $m_{\text{loc}} \cdot n_{\text{loc}} \cdot k$ elementi contro i $m_{\text{loc}} \cdot n_{\text{loc}}$ di A . X è piccola e sta in L2, quindi il traffico non arriva in DRAM; ma è banda L2, e cresce linearmente con k . È la trasposizione esatta, sulla gerarchia della GPU, dello stesso fenomeno descritto per la CPU in sezione 4.5.

Il rimedio. Gli 8 warp di uno stesso blocco lavorano su righe *diverse* di A , ma allo stesso passo j vogliono le *stesse* righe di X . Il blocco stagia quindi cooperativamente un tile $\text{x_rows_per_tile} \times k$ di X in shared memory e i suoi warp lo riusano: il traffico verso L2 si divide per il numero di warp per blocco.

$$\text{traffico L2 su } X : \underbrace{m_{\text{loc}} n_{\text{loc}} k}_{\text{cuda_warp}} \longrightarrow \underbrace{\frac{m_{\text{loc}} n_{\text{loc}} k}{W}}_{\text{cuda_warp_smem}}, \quad W = \text{warp per blocco.} \quad (5.3)$$

Ipotesi: il guadagno può essere nullo, e sarebbe comunque un risultato

Se in FP64 il kernel è già compute-bound — come il roofline di sezione 6.1 suggerisce per $k \gtrsim 4$ — ridurre il traffico L2 non può dare alcun guadagno. La misura serve a dimostrarlo, non a nascondere: un guadagno nullo qui è una conferma del modello, e va riportato come tale. La stessa misura in singola precisione, dove il modello prevede un regime memory-bound, è il controllo incrociato naturale.

5.4.1 Il conflitto sui banchi e il padding a $k + 1$

La shared memory ha 32 banchi da 32 bit. Un `double` ne occupa **due**. La lane l legge `X_tile[l*tile_row_stride + c]`, quindi fra due lane consecutive ci sono `tile_row_stride` scalari, cioè $2 \cdot \text{tile_row_stride}$ banchi:

<code>tile_row_stride</code>	banchi di stride	effetto
$k = 32$	$64 \equiv 0 \pmod{32}$	tutte e 32 le lane sullo stesso banco: conflitto a 32 vie
$k + 1 = 33$	$66 \equiv 2 \pmod{32}$	stride dispari in <code>double</code> : conflitto minimo

È l'analogo esatto del padding $\text{LDA} = N + 24$ studiato sulla CPU contro i conflict miss di L1: stesso fenomeno, altra gerarchia di memoria. Il padding è un parametro di compilazione (`SMEM_PAD`) proprio perché i due casi vanno *misurati* entrambi, e i due binari coesistono (??): non c'è modo di misurare per sbaglio la build sbagliata.

5.4.2 Che cosa il tile cambia davvero, e che cosa no

Il tile pesa $\text{x_rows_per_tile} \cdot (k + \text{SMEM_PAD}) \cdot s$ byte e k è noto solo a runtime: la shared memory è quindi dinamica e il numero di righe si sceglie in `local_gemm_create`, una volta sola, fuori dal cammino cronometrato.

Il punto da chiarire prima di tutto è quale grandezza il tile governi, perché l'intuizione naturale è sbagliata:

Il tile non cambia il traffico verso L2

Il fattore di riuso è il numero di warp per blocco, e basta. Ogni blocco legge comunque tutta la sua fetta di X una volta, che lo faccia in due tile grandi o in venti piccoli: il traffico resta $m_{\text{loc}} n_{\text{loc}} k / W$ in ogni caso.

Ciò su cui il tile agisce davvero è il **numero di barriere** e l'ILP. Ogni tile costa due `__syncthreads()`, e un tile che contiene un solo passo di warp (`x_rows_per_tile = 32`) fa arrivare ogni warp alla barriera successiva dopo una sola iterazione del ciclo su j , senza nessuna iterazione con cui coprire la latenza delle letture dalla shared memory. Dimezzare il numero di tile dimezza le barriere; non tocca il traffico.

Il budget non si sceglie in assoluto. La domanda giusta non è “quanti KiB dare a un blocco” ma “quanti gliene servono perché la shared *non sia lei* il vincolo attivo sull'occupancy”. La differenza è sostanziale: a $k = 32$ i 32 accumulatori costano già 64 registri a 32 bit per thread, quindi il vincolo attivo sono quasi certamente i registri — e un budget di shared scelto a mano più stretto starebbe rimpicciolendo il tile per proteggere un'occupancy già persa altrove.

La funzione `plan_tile` risolve il problema dell'uovo e della gallina — l'API dell'occupancy vuole i byte di shared che stiamo calcolando — interrogandola con `smem = 0`:

1. `cudaOccupancyMaxActiveBlocksPerMultiprocessor` con shared memory “gratis” dà quanti blocchi consentono *registri e thread per SM da soli*: è l'obiettivo da non peggiorare;
2. il budget per blocco è la shared memory per SM diviso quell'obiettivo (oppure il valore forzato da `SMEM_BUDGET_BYTES`), limitato dal tetto architetturale di 48 KiB oltre il quale il *lancio* fallirebbe;
3. le righe che ci stanno vengono arrotondate per difetto a `TILE_GRANULARITY`, con un minimo di un passo di warp completo e senza mai allocare più righe di quante X ne abbia davvero;

4. il piano viene **verificato invece che creduto**: il driver riserva shared memory per blocco oltre a quella richiesta, quindi il passo 2 può essere ottimista. Si reinterroga l'API con i byte effettivi e si scende di un passo finché l'obiettivo è raggiunto. La riserva del driver non viene modellata con una costante inventata: viene scoperta.

Il piano risultante finisce nel CSV come `x_rows_per_tile` e `blocks_per_sm`. Non sono metriche di prestazione: sono la configurazione effettivamente scelta, e senza di esse una campagna sul tiling mostrerebbe curve che salgono e scendono senza poter distinguere un cambio di tile da un cambio di occupancy.

La granularità è un compromesso, non un requisito. `TILE_GRANULARITY` vale 32 per default, ma non è una condizione di correttezza: il kernel gestisce già i tile parziali, sia nel caricamento cooperativo sia nel ciclo di consumo. È soltanto l'ottimizzazione “nessuna lane inattiva nell'ultimo passo di warp”, e ha un costo misurabile: a $k = 32$ in doppia precisione le righe che entrano nel budget sono 62, e arrotondare a 32 ne butta via metà, raddoppiando le barriere per guadagnare zero lane. Da qui il parametro: costo e beneficio si misurano, non si assumono.

Per lo stesso motivo non esiste più un massimo espresso in *righe*: non era un vincolo fisico e, appena il budget cresce, sarebbe diventato silenziosamente il vincolo attivo ai k piccoli, rendendo inefficace qualunque sweep sul budget. Il solo tetto rimasto è quello architetturale in byte.

5.4.3 Perché non ci sono uscite anticipate

`cuda_warp` può fare `if (warp_id >= m) return;` in cima. Qui non si può: il ciclo sui tile contiene `__syncthreads()`, che è una barriera di *blocco*. Se i warp dell'ultimo blocco — quello parzialmente pieno — uscissero prima, i rimanenti aspetterebbero a una barriera che nessuno raggiunge più. Tutti i thread del blocco percorrono quindi lo stesso numero di iterazioni e di barriere; sono soltanto l'accumulo e la scrittura finale a essere condizionati da un flag `active`.

5.5 `cuda_warp_tiled`: meno accumulatori per lane

L'ultima variante sposta il compromesso nella direzione opposta: un warp calcola un tile di `WARP_COL_TILE` colonne di una sola riga, invece di tutte e k . Le lane continuano a visitare $j = l, l + 32, \dots$ come prima.

Detto $T = \text{WARP_COL_TILE}$:

$$\text{warp per riga} = \left\lceil \frac{k}{T} \right\rceil, \quad \text{accumulatori per lane} = T, \quad \text{letture di } A \times \left\lceil \frac{k}{T} \right\rceil. \quad (5.4)$$

Il tile è un parametro di compilazione (`WARP_COL_TILE`, default 8) e compare nel nome del kernel, quindi lo sweep sui valori 4, 8, 16, 32 produce righe distinguibili nello stesso CSV. È la variante che serve a rispondere a una domanda precisa: per $k = 32$, tenere 32 accumulatori per lane costa in occupancy più di quanto costi rileggere A quattro volte? Il modello non risponde da solo, perché i due effetti hanno segno opposto; la campagna C2 lo misura.

5.6 `cublas`: il riferimento esterno

Non è una proposta, è la risposta a “perché avete scritto un kernel invece di chiamare una libreria?”. La risposta ha valore solo se il numero della libreria è stato misurato sulla stessa macchina, con la stessa pipeline, gli stessi trasferimenti e lo stesso cronometro: per questo

cuBLAS entra come un backend qualsiasi, dietro la stessa interfaccia, e non come uno script a parte.

5.6.1 Riga contro colonna: nessuna trasposizione

cuBLAS è column-major, il progetto è row-major. La soluzione non è trasporre — costerebbe più del prodotto — ma reinterpretare: una matrice $P \times Q$ row-major con leading dimension ld è *già*, bit per bit, la matrice $Q \times P$ column-major con la stessa ld . Leggere gli stessi byte con l'altra convenzione equivale a trasporre gratis. Quindi

$$Y_{cm} = (Y_{rm})^T = (A_{rm} X_{rm})^T = (X_{rm})^T (A_{rm})^T = X_{cm} A_{cm}, \quad (5.5)$$

e nella convenzione $C(M \times N) = A(M \times K) B(K \times N)$ la chiamata diventa $M = k$, $N = m_{loc}$, $K = n_{loc}$, con dX e dA in quest'ordine ed entrambe le operazioni a CUBLAS_OP_N. I vincoli di cuBLAS sulle leading dimension si traducono in $ldx \geq k$, $lda \geq n_{loc}$, $ldy \geq k$: esattamente le tre condizioni che `local_gemm_create` già verifica.

L'handle cuBLAS è creato in `local_gemm_create`: la sua creazione è costosa perché carica i kernel della libreria, e metterla nel cammino misurato misurerebbe l'inizializzazione della libreria invece del suo GEMM.

5.6.2 L'attesa è che vada male

cuBLAS è un GEMM general-purpose, ottimizzato per matrici in cui tutte e tre le dimensioni sono grandi. Qui $k \leq 32$: la libreria non ammortizza il proprio percorso di selezione dell'algoritmo, e il tiling che sceglie ha molte più righe che colonne utili. Un numero basso in questa riga è l'argomento più forte a sostegno dell'intero progetto — purché sia misurato, ed è ciò che C2 fa.

Da completare con i dati della campagna

Profilare con `nsys` o `ncu` quale kernel interno cuBLAS seleziona per $k \in \{3, 8, 32\}$, e verificare empiricamente la geometria del tile scelto. È il dato che trasforma “va male” in “va male *perché*”.

5.7 Più rank MPI sulla stessa GPU

Con più rank MPI sullo stesso nodo i contesti CUDA si alternano sulla scheda, perché MPS non è attivo. Il risultato resta corretto, ma t_{kernel} comprende anche il tempo in cui il contesto di un rank è sospeso a favore di un altro.

La conseguenza sulla lettura dei numeri è netta, e va enunciata prima di presentarli: **con $P > 1$ su una sola GPU, t_{kernel} non è il tempo di una GPU dedicata, ma il throughput aggregato della scheda sul problema globale.** Per caratterizzare un kernel in sé — confronto fra backend, sweep su BLOCK, effetto del padding in shared memory — va usato `-np 1`, che sulla stessa scheda esegue lo stesso lavoro totale senza contesa fra contesti. Tutte le campagne di caratterizzazione del kernel (C2, C3, C1 nella variante GPU) sono infatti a rank singolo.

Da completare con i dati della campagna

Verificare sul nodo di calcolo (non sul frontend) il supporto CUDA-aware di Open MPI con `ompi_info | grep cuda` e il numero di GPU con `nvidia-smi`. Se il supporto è presente, l'interfaccia `local_gemm` non richiede riscritture per passare puntatori device alle collettive: è stata progettata agnostica rispetto alla posizione della memoria proprio per questo, ed è un lavoro futuro (sezione 10.3).

Capitolo 6

Modello roofline

Il modello roofline mette in relazione l'intensità aritmetica di un nucleo con le due sole cose che possono limitarlo, la potenza di calcolo e la banda di memoria:

$$P_{\text{ottenibile}}(I) = \min(P_{\text{picco}}, B \cdot I), \quad I_{\text{crossover}} = \frac{P_{\text{picco}}}{B}. \quad (6.1)$$

Serve qui a due scopi: dire *prima* della misura quale regime aspettarsi per ogni k , e dare un metro con cui giudicare i numeri ottenuti. Un kernel che raggiunge il 70% del tetto pertinente è un buon kernel; lo stesso numero assoluto, confrontato con il tetto sbagliato, non dice nulla.

Dalla eq. (2.3), con lo schema A e contando il solo traffico su A :

$$I(k) = \frac{2k}{s} = \begin{cases} k/4 & \text{FP64 } (s = 8) \\ k/2 & \text{FP32 } (s = 4) \end{cases} \quad (6.2)$$

Tabella 6.1: Intensità aritmetica del nucleo, in FLOP/B, per i k della traccia.

	$k = 3$	$k = 6$	$k = 8$	$k = 20$	$k = 32$
FP64	0,75	1,50	2,00	5,00	8,00
FP32	1,50	3,00	4,00	10,00	16,00

6.1 Roofline della GPU

Da completare con i dati della campagna

I valori di picco in tabella 6.2 sono quelli nominali della Quadro RTX 5000 (TU104). Confermarli sul nodo di calcolo con `deviceQuery` (clock, numero di SM, larghezza e frequenza del bus di memoria) e, per la banda effettivamente raggiungibile, con una misura tipo STREAM su device: il rapporto fra banda nominale e banda misurata è tipicamente 0,75–0,85, e usare quella misurata sposta il punto di crossover.

Tabella 6.2: Picchi nominali della GPU di misura e punti di crossover derivati.

Grandezza	Valore	Origine
CUDA core	3072	specifica TU104
Clock boost	1,815 GHz	specifica
Picco FP32	11,1 TFLOP/s	$2 \times 3072 \times f$
Picco FP64	0,35 TFLOP/s	FP32/32 (rapporto Turing)
Banda di memoria	448 GB/s	GDDR6, 256 bit, 14 Gbit/s
Memoria	15,5 GiB	disponibile
Crossover FP64	0,78 FLOP/B	P_{picco}/B
Crossover FP32	24,9 FLOP/B	P_{picco}/B

Il confronto fra tabella 6.1 e tabella 6.2 dà la previsione, ed è il risultato più utile di tutto il modello:

Previsione

- **In FP64** il crossover cade a $I = 0,78$, cioè a $k \approx 3,1$. Il nucleo è quindi *memory-bound solo per $k = 3$* — e appena — e **compute-bound per ogni $k \geq 6$** . Il tetto pertinente per i quattro k più grandi è 350 GFLOP/s, e non dipende da k : le curve GFLOPS(k) dei backend GPU in doppia precisione dovrebbero appiattirsi.
- **In FP32** il crossover cade a $I = 24,9$, cioè a $k \approx 50$. Il nucleo è quindi **memory-bound per tutti i k della traccia**, e il tetto vale $B \cdot k/2 = 224k$ GFLOP/s: cresce *linearmente* con k e per $k = 32$ arriva a 7,2 TFLOP/s, venti volte il tetto FP64.

Le due precisioni si trovano quindi in regimi *qualitativamente* diversi, e questo è l'argomento quantitativo con cui il progetto discute la scelta della precisione: la doppia non costa il fattore 2 del traffico, costa il fattore 32 del picco FP64 di una scheda Turing. La previsione ha una conseguenza direttamente misurabile, ed è quella che la campagna C7 (??) mette alla prova: il rapporto fra i tempi nelle due precisioni deve **crescere con k** . Se fosse piatto e vicino a 2, sarebbe questo modello a essere sbagliato.

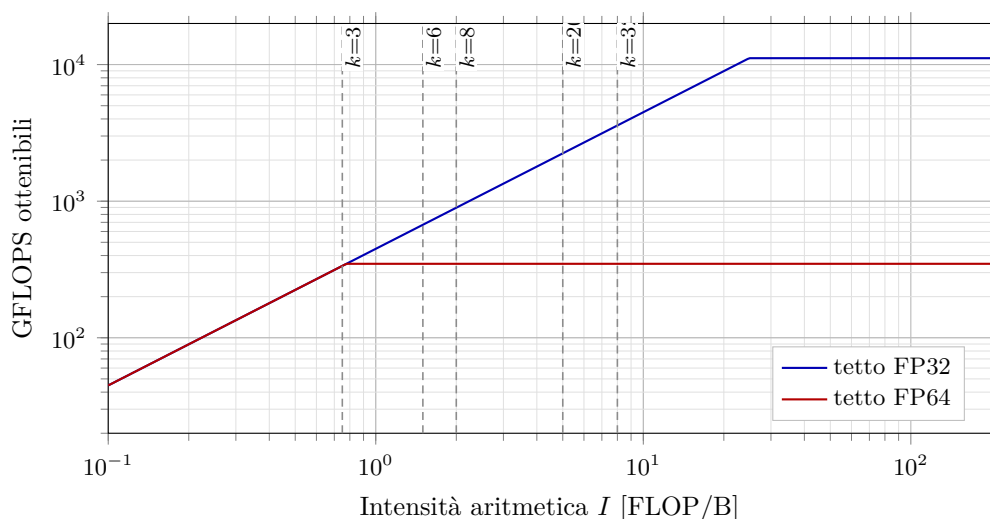


Figura 6.1: Roofline della GPU di misura. Le verticali tratteggiate sono le intensità aritmetiche FP64 dei cinque k della traccia. In FP64 solo $k = 3$ cade a sinistra del ginocchio; in FP32 tutti i k della traccia restano nella regione limitata dalla banda.

6.1.1 Che cosa il modello non dice

Due limiti vanno enunciati, perché altrimenti il modello verrebbe usato per concludere più di quanto possa.

Il primo: l'intensità della eq. (6.2) conta il solo traffico su A verso la DRAM. Il traffico su X , che vale $m_{\text{loc}} n_{\text{loc}} k$ elementi (eq. (4.2)), non compare perché X sta in L2 — ma esiste, e consuma banda L2. Il roofline classico a un solo livello non lo vede; è esattamente il traffico che `cuda_warp_smem` riduce (sezione 5.4), e la ragione per cui un kernel può restare sotto il tetto pur avendo l'intensità giusta.

Il secondo: il tetto di calcolo presuppone che ci sia abbastanza parallelismo per saturare gli SM. Con un warp per riga di Y , i warp disponibili sono m_{loc} : una matrice larga e bassa (rapporto $M:N$ sfavorevole) può affamare la GPU di parallelismo pur avendo la stessa intensità aritmetica. È precisamente il motivo per cui la campagna C1 varia il rapporto $M:N$ a *lavoro costante*: separa l'effetto della forma da quello della taglia.

6.2 Roofline della CPU

Da completare con i dati della campagna

Completare tabella 6.3 con i dati del server: modello del processore, numero di core fisici per socket, frequenza sostenuta con vettorizzazione attiva, larghezza SIMD, dimensione di L1/L2/L3, banda misurata con STREAM a un core e a tutti i core. Il modello del nucleo di CPU non è completabile senza la dimensione di L2, perché la soglia discussa in sezione 4.5 è proprio quella.

Tabella 6.3: Parametri del nodo di calcolo (da completare).

Grandezza	Valore	Come ottenerlo
Modello CPU	—	<code>lscpu</code>
Socket $\times$ core fisici	$2 \times$ —	<code>lscpu</code>
Frequenza sostenuta	—	<code>turbostat</code> o <code>perf</code>
Larghezza SIMD	—(AVX2 / AVX-512)	<code>lscpu grep Flags</code>
L1d / L2 per core, L3 per socket	—	<code>lscpu -C</code>
Banda STREAM, 1 core	—	STREAM triad
Banda STREAM, tutti i core	—	STREAM triad
Picco FP64, 1 core	—	$2 \times \text{SIMD} \times \text{FMA} \times f$

Anche senza questi valori, i dati già raccolti in campagna C3 permettono di ricavare per via inversa le grandezze rilevanti, ed è ciò che sezione 9.2 fa: dal tempo misurato si ottiene la banda effettiva su A e quella su X , e il confronto fra le due individua il livello di gerarchia che sta limitando il nucleo per ciascun k .

Capitolo 7

Metodologia sperimentale

7.1 Ambiente sperimentale

Le campagne sperimentali sono state eseguite sul server di calcolo del Dipartimento. La configurazione hardware e software rilevante ai fini delle misure è riassunta in tabella 7.1.

Tabella 7.1: Configurazione dell'ambiente sperimentale.

Componente	Configurazione
CPU	[DA FARE: modello CPU]
Socket / core	[DA FARE: numero socket e core]
Memoria principale	[DA FARE: quantità di RAM]
GPU	NVIDIA Quadro RTX 5000
Memoria GPU	15,56 GiB
Compute Capability	7.5 (Turing)
Streaming Multiprocessor (SM)	48
Compilatore GNU	GCC 13.3.0
Open MPI	5.0.9
CUDA Toolkit	12.8
Architettura CUDA	sm_75

L'ambiente software viene predisposto tramite il sistema di moduli del server, utilizzando versioni fissate della toolchain GNU, di Open MPI e di CUDA:

```
module purge
module load gnu/13.3.0
module load openmpi/5.0.9
module load cuda/12.8
```

In questo modo tutte le campagne vengono compilate ed eseguite utilizzando la stessa toolchain software.

Le implementazioni CPU sono compilate con ottimizzazione `-O3` e ottimizzazioni specifiche per l'architettura host tramite `-march=native`; viene inoltre abilitata la contrazione delle operazioni floating-point mediante `-ffp-contract=fast`. Le implementazioni CUDA sono compilate con `-O3` per l'architettura `sm_75`, corrispondente alla Compute Capability 7.5 della GPU utilizzata. La precisione numerica, singola o doppia, viene selezionata a tempo di compilazione.

Per le esecuzioni MPI multi-processo i rank vengono vincolati ai core mediante le opzioni `-bind-to core -map-by core`. Il placement esplicito riduce la variabilità dovuta alla migrazione dei processi e rende più controllabile la località degli accessi alla memoria durante il confronto tra configurazioni.

Il nodo dispone di una singola GPU e tutti i rank CUDA utilizzano il device 0. Di conseguenza, quando più rank MPI eseguono contemporaneamente un backend CUDA, essi condividono la stessa GPU e i tempi del kernel possono includere effetti di contesa e alternanza tra i diversi contesti CUDA. Per questo motivo le campagne finalizzate al confronto diretto tra kernel GPU e alla caratterizzazione della loro efficienza computazionale vengono eseguite con un solo rank MPI ($P = 1$). Le eventuali esecuzioni CUDA con $P > 1$ descrivono invece il comportamento del sistema in presenza di condivisione della singola GPU e non uno strong scaling su più acceleratori.

Per ogni campagna il runner registra inoltre nei metadati il comando eseguito, i principali parametri sperimentali, il commit Git, lo stato del working tree, le versioni di CUDA e Open MPI e le informazioni sulla GPU. Tali informazioni consentono di associare ogni insieme di misure alla configurazione software con cui è stato prodotto.

7.2 Protocollo di misura

Le misure vengono raccolte durante l'esecuzione del prodotto distribuito, separando le tre fasi principali dell'algoritmo: distribuzione del blocco locale di X , calcolo locale e riduzione dei contributi di Y .

Prima di ogni repetition misurata viene eseguita una barriera sul comunicatore della griglia MPI, in modo che tutti i rank inizino l'operazione dalla stessa fase dell'esecuzione. La barriera precede l'avvio del cronometro e non è quindi inclusa nei tempi riportati.

Per ciascun rank, una repetition segue la sequenza

$$\text{MPI_Bcast} \longrightarrow \text{local_gemm} \longrightarrow \text{MPI_Reduce}. \quad (7.1)$$

Il broadcast rende disponibile a tutti i processi interessati il blocco di X necessario al calcolo locale; il risultato parziale prodotto da `local_gemm` viene successivamente combinato mediante la reduce. Le due operazioni collettive utilizzano rispettivamente i comunicatori di colonna e di riga della griglia bidimensionale.

I quattro istanti temporali vengono acquisiti mediante `MPI_Wtime`:

$$t_0 : \text{inizio del broadcast}, \quad (7.2)$$

$$t_1 : \text{fine del broadcast}, \quad (7.3)$$

$$t_2 : \text{fine della fase locale}, \quad (7.4)$$

$$t_3 : \text{fine della reduce}. \quad (7.5)$$

Da essi vengono ricavati i tempi

$$t_{\text{bcast}} = t_1 - t_0, \quad (7.6)$$

$$t_{\text{local}} = t_2 - t_1, \quad (7.7)$$

$$t_{\text{reduce}} = t_3 - t_2, \quad (7.8)$$

$$t_{\text{total}} = t_3 - t_0. \quad (7.9)$$

Il tempo t_{total} rappresenta quindi il tempo wall-clock dell'intera invocazione distribuita, dall'inizio del broadcast alla conclusione della reduce. Le operazioni di preparazione del problema e di inizializzazione del backend vengono eseguite prima delle repetition e non appartengono a tale intervallo temporale.

Misura della fase CUDA. Nei backend CPU la fase `local_gemm` coincide con il calcolo locale. Nei backend CUDA, invece, la stessa fase comprende il trasferimento del blocco di X dalla memoria host alla GPU, l'esecuzione del kernel e il trasferimento del risultato locale Y dalla GPU alla memoria host:

$$t_{\text{local}} \simeq t_{\text{H2D},X} + t_{\text{kernel}} + t_{\text{D2H},Y} + t_{\text{overhead}}. \quad (7.10)$$

I tre intervalli GPU principali vengono misurati separatamente mediante CUDA events. In particolare vengono registrati il tempo del trasferimento H2D di X , il tempo di esecuzione del kernel e il tempo del trasferimento D2H di Y . La sincronizzazione associata al completamento dell'ultimo trasferimento garantisce che la fase locale sia conclusa prima che venga eseguita la successiva `MPI_Reduce`.

La copia iniziale della matrice locale A verso la memoria della GPU avviene invece durante l'inizializzazione del backend e non viene ripetuta a ogni invocazione del prodotto.

7.3 Definizione della metrica di prestazione

I tempi descritti nella sezione precedente vengono misurati separatamente da ciascun rank MPI. Poiché l'esecuzione distribuita può proseguire solo alla velocità del processo più lento, il tempo associato a una repetition viene definito considerando il massimo tra i rank.

Indicando con i la repetition e con p il rank MPI, il tempo ufficiale locale della repetition è definito in modo diverso per i backend CPU e CUDA.

Per i backend CPU:

$$T_{i,p}^{\text{CPU}} = t_{\text{bcast},i,p} + t_{\text{local},i,p} + t_{\text{reduce},i,p}. \quad (7.11)$$

Poiché in questo caso l'intera fase locale coincide con il calcolo eseguito dalla CPU, tale quantità corrisponde anche al tempo wall-clock t_{total} misurato dall'inizio del broadcast alla fine della reduce.

Per i backend CUDA la metrica richiesta esclude invece i trasferimenti tra memoria host e device. La fase locale viene pertanto rappresentata dal solo tempo di esecuzione del kernel:

$$T_{i,p}^{\text{CUDA}} = t_{\text{bcast},i,p} + t_{\text{kernel},i,p} + t_{\text{reduce},i,p}. \quad (7.12)$$

Il tempo rappresentativo della repetition i è quindi

$$T_i = \max_{0 \leq p < P} T_{i,p}, \quad (7.13)$$

dove P è il numero complessivo di processi MPI.

La composizione delle diverse fasi viene effettuata localmente prima dell'operazione di massimo. In particolare, nel caso CUDA viene calcolato

$$\max_p (t_{\text{bcast},p} + t_{\text{kernel},p} + t_{\text{reduce},p}), \quad (7.14)$$

e non la somma

$$\max_p t_{\text{bcast},p} + \max_p t_{\text{kernel},p} + \max_p t_{\text{reduce},p}. \quad (7.15)$$

La seconda espressione potrebbe infatti combinare tempi appartenenti a rank differenti e non rappresenterebbe il percorso temporale di alcun processo reale.

Dopo aver ottenuto un valore T_i per ciascuna delle R repetition misurate, il tempo medio utilizzato per valutare le prestazioni è

$$\bar{T} = \frac{1}{R} \sum_{i=1}^R T_i. \quad (7.16)$$

Il prodotto tra una matrice $A \in \mathbb{R}^{M \times N}$ e un multivettore $X \in \mathbb{R}^{N \times k}$ richiede approssimativamente

$$2MNk \quad (7.17)$$

operazioni floating-point. La prestazione viene quindi espressa come

$$\text{GFLOP/s} = \frac{2MNk}{\bar{T}} \cdot 10^{-9}. \quad (7.18)$$

Questa quantità costituisce la metrica principale utilizzata per il confronto tra le diverse configurazioni sperimentali.

Metriche ausiliarie. Oltre alla metrica complessiva, il programma calcola una misura della sola fase computazionale. Per i backend CPU essa utilizza t_{local} , mentre per i backend CUDA utilizza t_{kernel} :

$$T_{\text{compute}} = \begin{cases} t_{\text{local}}, & \text{CPU,} \\ t_{\text{kernel}}, & \text{CUDA.} \end{cases} \quad (7.19)$$

La corrispondente prestazione è riportata come `gflops_compute`. Nei backend CUDA la stessa quantità è riportata anche come `gflops_kernel`.

La colonna `gflops` rimane invece la metrica di riferimento, poiché comprende anche il costo delle comunicazioni MPI previste dal prodotto distribuito.

7.4 Costi esclusi dalla metrica ufficiale

La metrica di prestazione definita nella sezione precedente considera esclusivamente il prodotto distribuito vero e proprio. Restano quindi esclusi dal tempo ufficiale i costi di preparazione del problema, l'I/O e, nel caso dei backend CUDA, le operazioni di trasferimento tra memoria host e memoria device.

In particolare, il preprocessing comprende l'allocazione dei buffer locali, la generazione o distribuzione iniziale dei dati, l'eventuale conversione del layout di X e l'inizializzazione del backend. Tali operazioni vengono eseguite prima delle repetition misurate e non contribuiscono quindi a T_{official} .

Per i backend CUDA, anche il trasferimento iniziale della matrice locale A verso la memoria della GPU appartiene alla fase di setup e viene eseguito una sola volta. Durante ciascuna invocation di `local_gemm` vengono invece eseguiti il trasferimento H2D del blocco di X , il kernel CUDA e il trasferimento D2H del risultato locale Y .

La metrica ufficiale utilizza, per la fase locale CUDA, il solo tempo del kernel. Di conseguenza i trasferimenti H2D e D2H, insieme all'eventuale overhead residuo lato host/runtime, non entrano in T_{official} . Essi vengono tuttavia misurati separatamente, così da poter distinguere il costo computazionale del kernel dal costo complessivo necessario a utilizzare l'acceleratore.

Il programma registra inoltre il tempo wall-clock completo della fase locale CUDA:

$$t_{\text{local}} \simeq t_{\text{H2D},X} + t_{\text{kernel}} + t_{\text{D2H},Y} + t_{\text{overhead}}, \quad (7.20)$$

dove t_{overhead} rappresenta un termine residuo ottenuto per differenza e comprende costi host/runtime non attribuiti direttamente ai tre intervalli misurati mediante CUDA events. Tale quantità non deve quindi essere interpretata come una misura diretta del solo costo di kernel launch.

Anche l'I/O necessario alla produzione dei file di output viene mantenuto fuori dai tempi del prodotto. In questo modo le scritture su file non influenzano le misure di prestazione, pur rimanendo disponibili per la successiva analisi dei risultati.

La separazione tra tempo ufficiale e costi accessori consente quindi di utilizzare una metrica coerente per il confronto tra implementazioni, senza rinunciare alla possibilità di valutare il costo reale del trasferimento dei dati e dell'inizializzazione dell'acceleratore.

7.5 Ripetizioni e stabilità statistica

Ogni configurazione sperimentale viene eseguita più volte all'interno della stessa esecuzione del programma. Prima delle repetition utilizzate per le statistiche vengono effettuate alcune invocazioni di warm-up, i cui tempi non vengono registrati. Il numero di warm-up e di repetition è definito esplicitamente nella configurazione della singola campagna.

Le invocazioni di warm-up hanno lo scopo di portare l'esecuzione in una condizione più rappresentativa prima della raccolta dei campioni, evitando che la prima misura sia influenzata da costi transitori. Al termine dei warm-up vengono eseguite le R repetition misurate secondo il protocollo descritto nelle sezioni precedenti.

Prima di ciascuna repetition viene eseguita una `MPI_Barrier` sul comunicatore della griglia. La barriera è esterna all'intervallo cronometrato e serve esclusivamente ad allineare l'inizio della nuova repetition tra i rank.

Per ciascuna repetition viene ottenuto il tempo globale T_i definito in eq. (7.13). L'insieme

$$\{T_1, T_2, \dots, T_R\} \quad (7.21)$$

costituisce il campione utilizzato per l'analisi statistica della configurazione.

Il valore principale riportato nei confronti prestazionali è la media aritmetica

$$\bar{T} = \frac{1}{R} \sum_{i=1}^R T_i. \quad (7.22)$$

Alla media vengono affiancate misure di dispersione e statistiche descrittive, tra cui deviazione standard campionaria, mediana e minimo. La deviazione standard è calcolata come

$$s = \sqrt{\frac{1}{R-1} \sum_{i=1}^R (T_i - \bar{T})^2}. \quad (7.23)$$

Per confrontare la variabilità di misure aventi scale temporali differenti viene inoltre utilizzato il coefficiente di variazione

$$CV = 100 \frac{s}{\bar{T}}, \quad (7.24)$$

espresso in percentuale.

Il numero di repetition non viene considerato, da solo, una garanzia di stabilità della misura. La dispersione viene invece verificata a posteriori attraverso il coefficiente di variazione e, nei casi in cui questo evidenzia una variabilità significativa, mediante l'ispezione dei singoli campioni. Differenze prestazionali dello stesso ordine della dispersione osservata non vengono considerate sufficienti, da sole, per trarre conclusioni sul confronto tra due configurazioni.

Oltre ai valori aggregati, il programma può produrre un file CSV contenente una riga per ciascuna repetition misurata. I campioni raw consentono di individuare eventuali outlier, variazioni sistematiche nel corso dell'esecuzione o altri comportamenti che potrebbero essere nascosti dalla sola media.

Le invocazioni di warm-up non vengono incluse nei file raw né nelle statistiche finali.

Nelle campagne principali vengono utilizzate tipicamente 20 repetition precedute da 5 warm-up; per le campagne con un numero maggiore di configurazioni vengono impiegati, dove previsto, 10 campioni dopo 3 warm-up. I valori effettivi sono comunque parte della configurazione della singola campagna e vengono registrati nei relativi metadati.

7.6 Disegno delle campagne sperimentali

Le campagne sperimentali sono organizzate in modo da isolare, per quanto possibile, l'effetto delle principali variabili del problema: dimensione e forma della matrice, numero di colonne del multivettore, implementazione del kernel locale, numero di processi MPI, forma della griglia bidimensionale e precisione numerica.

Ogni campagna modifica quindi un insieme limitato di parametri mantenendo costanti gli altri. Le configurazioni utilizzate sono definite tramite file `.conf`, in modo da rendere espliciti e riproducibili i parametri di ogni esperimento.

7.6.1 C1 — Dimensione del problema e rapporto $M:N$

La prima campagna studia l'effetto della dimensione globale del problema e della forma della matrice. Vengono considerati tre rapporti tra le dimensioni:

$$M : N \in \{1 : 1, 3 : 1, 1 : 2\}, \quad (7.25)$$

inclusendo quindi sia matrici quadrate sia configurazioni rettangolari.

Per ciascun rapporto vengono utilizzate quattro taglie crescenti. I valori di M e N sono scelti in modo che, a parità di livello, il prodotto MN sia approssimativamente equivalente tra i tre rapporti. Ciò permette di confrontare problemi con una quantità di lavoro simile ma con una diversa geometria della matrice.

La campagna viene eseguita sia per il backend CPU sia per un backend CUDA, utilizzando un singolo processo MPI ($P = 1$), in modo da isolare gli effetti della dimensione e della forma del problema dalla comunicazione MPI. Vengono considerati $k = 8$ e $k = 32$, rappresentativi di due differenti dimensioni del multivettore.

7.6.2 C2 — Sweep su k e confronto dei backend

La seconda campagna confronta le diverse implementazioni mantenendo fisse le dimensioni del problema e variando il numero di colonne k del multivettore.

Sono innanzitutto considerati i valori richiesti dalla traccia, $k \in \{3, 6, 8, 20, 32\}$. A questi vengono aggiunti alcuni valori non specializzati, utilizzati per verificare il comportamento del percorso generico dell'implementazione.

Il problema viene mantenuto fisso a $M = N = 10000$ e viene utilizzato un singolo processo MPI ($P = 1$). In questo modo il confronto riflette principalmente le differenze tra le implementazioni locali, senza introdurre il costo delle comunicazioni tra processi.

Le implementazioni confrontate comprendono il backend CPU `scheme_a`, i kernel CUDA `cuda_naive`, `cuda_warp`, `cuda_warp_smem` e `cuda_warp_tiled`, oltre all'implementazione basata su cuBLAS, utilizzata come riferimento.

La campagna C3 è progettata come confronto controllato tra la versione specializzata e la versione generica del kernel CPU.

Le due configurazioni utilizzano gli stessi valori di M , N , k e lo stesso numero di processi; l'unica differenza rilevante è l'abilitazione o meno della specializzazione a compile-time per k .

Questo consente di attribuire le differenze osservate direttamente alla specializzazione, evitando il confronto tra problemi con una quantità di lavoro differente.

La campagna viene eseguita con $M = N = 10000$ e $P = 1$.

7.6.3 C4 — Strong scaling

La campagna C4 studia lo strong scaling dell'implementazione MPI.

La dimensione globale del problema viene mantenuta costante a

$$M = N = 20000, \quad (7.26)$$

mentre il numero di processi varia secondo

$$P \in \{1, 2, 4, 8, 16, 20\}. \quad (7.27)$$

Per ogni valore di P viene utilizzata una griglia bidimensionale prossima alla forma quadrata, in modo da evitare di introdurre intenzionalmente configurazioni fortemente sbilanciate.

Vengono considerati $k = 3$ e $k = 32$, così da osservare il comportamento dello scaling per due differenti dimensioni del multivettore.

Lo speedup di strong scaling viene calcolato rispetto alla configurazione a un processo della stessa campagna:

$$S(P) = \frac{T(1)}{T(P)}, \quad (7.28)$$

mentre l'efficienza parallela è

$$E(P) = \frac{S(P)}{P}. \quad (7.29)$$

7.6.4 C5 — Weak scaling

La campagna C5 studia il weak scaling mantenendo approssimativamente costante la quantità di dati assegnata a ciascun processo.

Per una griglia $P_r \times P_c$ vengono scelte le dimensioni globali secondo

$$M = 5000P_r, \quad N = 5000P_c. \quad (7.30)$$

Di conseguenza ciascun processo opera, salvo gli effetti della decomposizione, su un blocco locale di circa

$$5000 \times 5000. \quad (7.31)$$

All'aumentare del numero di processi cresce quindi la dimensione globale del problema, mentre il lavoro computazionale locale nominale rimane approssimativamente costante.

L'andamento del tempo totale e la sua scomposizione nelle componenti di broadcast, calcolo locale e reduce permettono di osservare come il costo delle comunicazioni e gli effetti del sistema evolvano all'aumentare di P .

7.6.5 C6 — Forma della griglia MPI

La campagna C6 isola l'effetto della forma della griglia bidimensionale.

Per un numero complessivo di processi fissato vengono considerate tutte le fattorizzazioni ordinate

$$P = P_r P_c. \quad (7.32)$$

Ad esempio, per $P = 8$ vengono confrontate le configurazioni

$$1 \times 8, \quad 2 \times 4, \quad 4 \times 2, \quad 8 \times 1. \quad (7.33)$$

La dimensione globale del problema e il numero complessivo di processi rimangono invariati all'interno di ciascun confronto; cambia solamente la ripartizione tra righe e colonne della griglia MPI.

La campagna consente di valutare sperimentalmente l'effetto della decomposizione bidimensionale sui costi di broadcast di X e reduce di Y , confrontando anche le configurazioni degeneri $1 \times P$ e $P \times 1$ con quelle maggiormente bilanciate.

7.6.6 C7 — Precisione numerica

L'ultima campagna studia l'effetto della precisione numerica sui backend CUDA. Il confronto viene effettuato tra FP64 e FP32 mantenendo invariati, per ogni coppia di misure, dimensione del problema, valore di k , kernel e configurazione di lancio.

La campagna è suddivisa in due parti complementari.

C7.1 — Prestazioni FP32 e FP64. I principali backend CUDA vengono compilati ed eseguiti sia in singola sia in doppia precisione, con $M = N = 10000$ e $P = 1$.

Il confronto viene effettuato utilizzando la stessa metrica temporale per le due precisioni. In particolare, quando l'obiettivo è analizzare il solo comportamento del kernel CUDA, vengono confrontati i rispettivi valori di t_{kernel} .

Lo speedup dovuto alla precisione può quindi essere espresso come

$$S_{32/64} = \frac{t_{\text{kernel}}^{FP64}}{t_{\text{kernel}}^{FP32}}. \quad (7.34)$$

C7.2 — Utilizzo delle risorse CUDA. La seconda parte non costituisce un benchmark temporale, ma una caratterizzazione delle risorse richieste dai kernel.

Le compilazioni vengono effettuate utilizzando l'output di `ptxas`, dal quale vengono ricavati il numero di registri per thread, eventuali spill, stack frame e shared memory statica.

Questi dati vengono utilizzati per correlare le differenze osservate tra FP32 e FP64 con la pressione sulle risorse hardware, senza considerarli una misura diretta dell'occupancy effettivamente raggiunta durante l'esecuzione.

Nel complesso, le campagne sono organizzate in modo da separare gli effetti delle diverse variabili: C1–C3 caratterizzano principalmente il comportamento computazionale delle implementazioni locali, C4–C6 studiano il comportamento della decomposizione MPI, mentre C7 approfondisce l'effetto della precisione numerica sui backend CUDA.

Le conclusioni relative alle prestazioni vengono formulate solo dopo l'analisi dei dati raccolti; le campagne definiscono quindi il protocollo di confronto, non il risultato atteso.

Una sintesi delle campagne è riportata in tabella 7.2.

Tabella 7.2: Sintesi delle campagne sperimentali.

ID	Variabile studiata	Obiettivo
C1	M , N , rapporto $M:N$	Valutare l'effetto della dimensione del problema e della forma della matrice su configurazioni CPU e GPU.
C2	k e backend	Confrontare le diverse implementazioni al variare del numero di colonne del multivettore e verificare il comportamento del percorso generico.
C3	specializzazione di k	Isolare il beneficio della specializzazione a compile-time confrontando, sullo stesso problema, versione specializzata e versione generica del kernel CPU.
C4	numero di processi P	Studiare lo strong scaling mantenendo costante la dimensione globale del problema.
C5	numero di processi P e dimensione globale	Studiare il weak scaling mantenendo costante la dimensione del blocco locale assegnato a ciascun processo.
C6	forma $P_r \times P_c$	Valutare l'effetto della forma della griglia MPI mantenendo costante il numero complessivo di processi.
C7	precisione FP32/FP64	Confrontare le due precisioni numeriche e correlare le prestazioni alle risorse utilizzate dai kernel CUDA.

Capitolo 8

Implementazione seriale, correttezza e validazione

La traccia chiede il collaudo funzionale contro un'implementazione seriale minimale. Il progetto la usa come oracolo, ma la strategia di correttezza è articolata su quattro livelli, perché i modi di sbagliare sono di natura diversa e un solo controllo non li copre.

8.1 L'oracolo seriale

`src/serial/serial.c` è la trascrizione letterale della eq. (2.1): tre cicli annidati nell'ordine $i \rightarrow c \rightarrow j$, cioè lo schema B. La scelta è deliberata: l'oracolo deve essere *ovviamente* corretto, non veloce, e usare un ordine dei cicli diverso da quello dei backend riduce la probabilità che un errore concettuale sull'ordinamento passi inosservato in entrambi.

Con `-check` il rank 0 raccoglie Y distribuita, ricostruisce da sé A e X globali attraverso il generatore (sezione 2.5) — senza riceverle da nessuno — calcola l'oracolo e confronta. Il costo è $O(MNk)$ seriale, che è proprio il motivo per cui la validazione non fa mai parte di una campagna di misura.

8.2 Una tolleranza che dipende da N

La tolleranza del confronto non è una costante, e non può esserlo: ogni elemento di Y è una riduzione su N termini, e l'errore di arrotondamento di una riduzione dipende da N .

Con dati a media nulla — che è esattamente ciò che il generatore produce, e per questo motivo — gli arrotondamenti hanno segno indipendente e l'errore cresce come $\varepsilon\sqrt{N}$, non come il pessimistico εN del caso in cui tutti i termini abbiano lo stesso segno. La costante del modello è **misurata**, non stimata: confrontando il prodotto distribuito con l'oracolo su $N = 257 \dots 40\,000$, in entrambe le precisioni, l'errore relativo in norma L_2 vale

$$\text{err} \approx 0,17 \cdot \varepsilon\sqrt{N}, \quad (8.1)$$

con un rapporto che resta fra 0,11 e 0,17 su tutto l'intervallo. La soglia adottata è quindi

$$\text{tol}(N) = 8 \cdot \varepsilon \cdot \sqrt{N}, \quad (8.2)$$

cioè un margine uniforme di circa $47\times$ sul modello, su tutte le taglie e in entrambe le precisioni.

Il confronto con una soglia fissa mostra perché la scelta conta, e sbaglia in due modi opposti proprio nell'intervallo di taglie che il progetto misura:

Soglia fissa	Difetto	Conseguenza
<code>double</code> , 10^{-12}	troppo larga	a $N = 257$ l'errore reale è $5,6 \cdot 10^{-16}$: la soglia è ~ 1800 volte l'errore, e a $N = 40\,000$ ancora ~ 140 volte. Un bug che gonfiasse l'errore di due ordini di grandezza passerebbe per PASS.
<code>float</code> , 10^{-5}	si restringe con N	il margine passa da $33\times$ a $N = 257$ a $2,5\times$ a $N = 40\,000$, che è la taglia a cui la campagna lavora. Basta un backend con un altro ordine di riduzione per produrre un FAIL su un risultato corretto — e siccome il binario esce con stato non nullo, quel FAIL abortisce la campagna.

Tabella 8.1: Perché una soglia costante non funziona in nessuna delle due precisioni.

Un ultimo dettaglio che conta: N nella eq. (8.2) è il numero *globale* di colonne di A , non n_{loc} , perché è su tutte e N che l'oracolo somma.

8.3 Il test delle funzioni indice

Da completare con i dati della campagna

La cartella `test/` non è presente nella copia di lavoro corrente, mentre il `Makefile` continua a costruire `bin/test_index` da `test/test_index.c` e la sonda `check-cxx` da `test/cxx_iface.cpp`: allo stato attuale `make`, `make test` e `make check-cxx` non possono funzionare. Ripristinare la cartella dal repository (`git checkout -- test/`) prima della consegna, oppure aggiornare questa sezione se i test sono stati rimossi di proposito.

Le tre funzioni di sezione 3.4 hanno un test isolato (`test/test_index.c`, `make test`) che non richiede MPI. Verifica su molte combinazioni (n, p) , inclusi i casi limite $p > n$ e $n \bmod p \neq 0$, tre proprietà:

- $\sum_i \text{size}(n, p, i) = n$: la partizione copre tutto;
- $\text{start}(n, p, i+1) = \text{start}(n, p, i) + \text{size}(n, p, i)$: le parti sono contigue e non si sovrappongono;
- `owner` è l'inversa di `start/size`: le due direzioni sono coerenti.

Sono i controlli che intercettano l'errore di un solo elemento, cioè proprio quello che si propagherebbe in silenzio in tutto il resto del sistema.

8.4 La matrice di collaudo

Tabella 8.2: I controlli eseguiti e che cosa ciascuno intercetta.

Comando	Che cosa protegge
<code>make test</code>	partizionamento degli indici, senza MPI
<code>make check-cxx</code>	<code>kernel.h</code> e <code>util.h</code> restano includibili da C++ e tutti i simboli dell'interfaccia conservano il linkage C (verificato con <code>nm</code>): intercetta le regressioni di <code>extern "C"</code> e <code>restrict</code> su qualunque macchina, anche senza <code>nvcc</code>
<code>make check</code>	prodotto distribuito contro oracolo seriale, doppia precisione
<code>make PREC=float check</code>	stessa validazione in singola precisione: è il caso in cui una tolleranza sbagliata si manifesta
<code>make check-padding</code>	<code>lda = n_{loc} + 8</code> : verifica che tutti i backend onorino la leading dimension di A
<code>make KERNEL=<nome> check</code>	la stessa validazione su ciascun backend GPU, che ha un ordine di riduzione diverso da quello di CPU
<code>make -B KERNEL=cuda_warp EXTRA_NVCCFLAGS='-Xptxas -v'</code>	ispezione dell'uso dei registri per warp: non è correttezza ma è il controllo che dice se la specializzazione su k sta facendo ciò che si crede

La validazione va eseguita su ogni combinazione di backend e precisione *prima* della campagna corrispondente, su taglie piccole. Un backend validato solo in doppia precisione non è validato: l'ordine della riduzione cambia con il kernel, e in singola precisione l'effetto è misurabile.

Da completare con i dati della campagna

Riportare l'esito effettivo dei controlli sul server, con il valore di `rel_err` misurato e la soglia corrispondente, per almeno: backend di CPU in `double` e `float`; `cuda_warp` e `cuda_warp_smem` in entrambe le precisioni; `cublas`; il caso con padding. Una tabella con colonne (backend, precisione, N , `rel_err`, `tol`, esito) è la forma più compatta.

Tabella 8.3: Esiti della validazione. **[DA FARE: compilare dopo l'esecuzione sul server]**

Backend	Precisione	N	<code>rel_err</code>	<code>tol</code>	Esito
<code>scheme_a</code>	double	–	–	–	–
<code>scheme_a</code>	float	–	–	–	–
<code>scheme_a</code> (padding)	double	–	–	–	–
<code>cuda_naive</code>	double	–	–	–	–
<code>cuda_warp</code>	double	–	–	–	–
<code>cuda_warp</code>	float	–	–	–	–
<code>cuda_warp_smem</code>	double	–	–	–	–
<code>cuda_warp_smem</code>	float	–	–	–	–
<code>cuda_warp_tiled</code>	double	–	–	–	–
<code>cublas</code>	double	–	–	–	–

Capitolo 9

Risultati

9.1 Stato della raccolta dati

Tabella 9.1: Stato delle campagne al momento di questa versione della relazione.

Campagna	Stato	Dati disponibili	Sezione
C3 (baseline CPU)	parziale	build specializzata, $P = 1$, 10000^2	sezione 9.2
C3 (build generica)	da eseguire	—	sezione 9.2.3
C2 (k -sweep, 6 backend)	da eseguire	—	sezione 9.3
C1 (taglia $\times$ rapporto)	da eseguire	—	sezione 9.4
C6 (forma della griglia)	da eseguire	—	sezione 9.5
C4 (strong scaling)	da eseguire	—	sezione 9.6
C5 (weak scaling)	da eseguire	—	sezione 9.7
C7.1 (precisione $\times k$)	da eseguire	—	sezione 9.8
C7.2 (risorse per precisione)	da eseguire	—	sezione 9.9

Da completare con i dati della campagna

Tempo macchina stimato per completare la raccolta: circa un'ora, escluse le compilazioni, nell'ordine C3 (build generica) $\rightarrow$ C2 $\rightarrow$ C1 $\rightarrow$ C6 $\rightarrow$ C4 $\rightarrow$ C5 $\rightarrow$ C7. Le sezioni che seguono sono già strutturate con le tabelle e le figure definitive: resta da sostituire i segnaposto con i numeri.

Va inoltre popolata sezione 9.10: i tempi esclusi dalla metrica ufficiale non richiedono nessuna esecuzione aggiuntiva, perché ogni campagna li scrive già in coda al proprio CSV.

9.2 C3 — baseline di CPU

Configurazione: `scheme_a`, doppia precisione, $M = N = 10\,000$, $P = 1$ (griglia 1×1), `-reps 20` `-warmup 5`, seed 20 252 026, generazione locale di A e X . Senza comunicazione e senza GPU: è il nucleo di calcolo puro, ed è il denominatore di ogni speedup della relazione.

Tabella 9.2: Campagna C3, build specializzata. t è il tempo ufficiale medio su 20 repetition; le ultime due colonne sono derivate, non misurate, e sono definite nel testo.

k	t [s]	CV [%]	GFLOPS	B_A [GB/s]	B_X [GB/s]
3	0,1336	1,38	4,49	5,99	18,0
6	0,1424	1,17	8,43	5,62	33,7
8	0,1473	1,10	10,87	5,43	43,5
20	0,7680	0,37	5,21	1,04	20,8
32	1,3073	0,43	4,90	0,61	19,6
<i>fallback generico (due passaggi su A, cfr. eq. (4.1))</i>					
40	1,7591	0,46	4,55	0,45	18,2
64	3,0332	0,37	4,22	0,26	16,9

Il coefficiente di variazione resta fra 0,37% e 1,38%: tutte le differenze discusse qui sotto sono di ordini di grandezza superiori alla dispersione delle misure. Il preprocessing del backend ($t_{\text{setup}} \approx 1,4 \mu\text{s}$) è trascurabile, come atteso per un backend che si limita a registrare un puntatore.

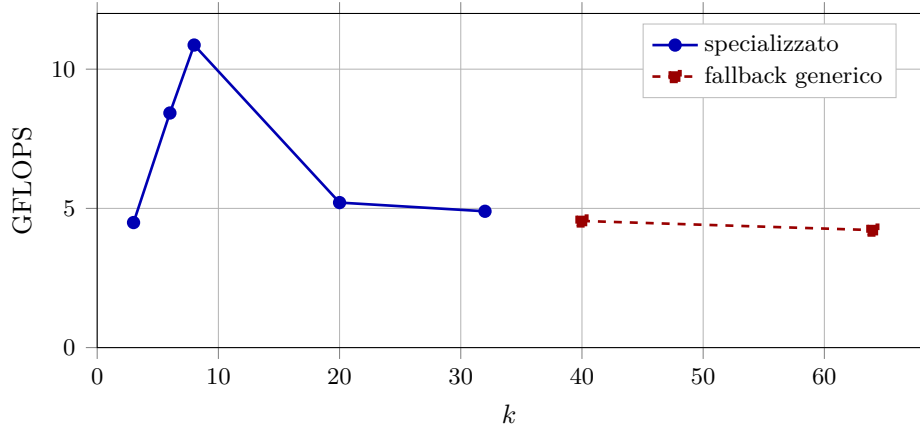


Figura 9.1: C3: prestazioni del nucleo di CPU al variare di k . Il massimo cade a $k = 8$; oltre, il throughput crolla.

9.2.1 Lettura: due regimi separati da una soglia di cache

La curva di fig. 9.1 ha una forma che non si spiega con il solo conteggio delle operazioni: fra $k = 8$ e $k = 20$ il lavoro cresce di $2,5\times$ ma il tempo cresce di $5,2\times$. Il modo più diretto di guardare il fenomeno è normalizzare rispetto al numero di elementi di A elaborati, che è costante:

$$\tau(k) = \frac{t(k)}{MN} \quad [\text{tempo per elemento di } A]. \quad (9.1)$$

Tabella 9.3: Costo per elemento di A e dimensione di X_{loc} .

k	τ [ns]	$\tau/\tau_{k=3}$	$ X_{\text{loc}} $
3	1,34	1,00	0,24 MB
6	1,42	1,07	0,48 MB
8	1,47	1,10	0,64 MB
20	7,68	5,75	1,60 MB
32	13,07	9,78	2,56 MB
40	17,59	13,17	3,20 MB
64	30,33	22,70	5,12 MB

Per $k \in \{3, 6, 8\}$ il costo per elemento di A è **quasi costante**: triplicare il numero di FMA per elemento aumenta il tempo del 10%. È la firma di un regime in cui il collo di bottiglia non è il calcolo ma lo streaming di A : la banda effettiva su A , $B_A = MNs/t$, vale da 5,4 GB/s a 6,0 GB/s, un valore plausibile per un singolo core, e le FMA aggiuntive si nascondono dietro le latenze di memoria.

Da $k = 20$ in poi il comportamento cambia natura. Se il limite fosse la banda su A , τ resterebbe piatto; se fosse il picco di calcolo, τ crescerebbe *proporzionalmente* a k . Non fa né l'una né l'altra cosa: da $k = 8$ a $k = 20$ cresce di $5,2\times$ per $2,5\times$ di lavoro. Il candidato che spiega il fattore è il traffico su X della eq. (4.2). Definendo

$$B_X = \frac{MNks}{t} \quad [\text{banda effettiva sulla riletture di } X], \quad (9.2)$$

la colonna corrispondente di tabella 9.2 mostra un valore sostanzialmente **costante** intorno a da 17 GB/s a 21 GB/s per tutti i $k \geq 20$ — inclusi $k = 40$ e $k = 64$, che passano dal fallback e leggono A due volte. Un plateau di banda su cinque punti che coprono un fattore 3,2 in k è una firma difficile da attribuire al caso.

Ipotesi: oltre $k \approx 8$ la riletture di X esce dalla cache privata

Nello schema A ogni riga di A percorre tutta X_{loc} . Finché X_{loc} sta nella cache privata del core, quelle riletture costano poco e il nucleo è limitato dallo streaming di A ($k \leq 8$: B_A costante, τ piatto). Quando X_{loc} supera quella capacità — fra 0,64 MB a $k = 8$ e 1,6 MB a $k = 20$ — ogni riga di A ricomincia a leggere X dal livello successivo della gerarchia, e il tempo diventa proporzionale a $MNks$ diviso la banda di quel livello: da qui il plateau di B_X .

Il valore del plateau, da 17 GB/s a 21 GB/s, è coerente con una banda di cache condivisa su singolo core, e nettamente superiore alla banda DRAM per core che si legge nel regime $k \leq 8$ (~ 6 GB/s): non si tratta quindi di traffico verso la memoria centrale.

Come falsificarla. (i) Leggere la capacità della cache privata con `lscpu -C` e verificare che cada fra 0,64 e 1,6 MB; (ii) contare i miss del livello sospetto con `perf stat -e` sui k ai due lati della soglia; (iii) ripetere C3 a N dimezzato: se l'ipotesi è giusta, la soglia in k deve raddoppiare, perché $|X_{\text{loc}}| = Nks$.

L'ipotesi alternativa — spill degli accumulatori oltre il numero di registri vettoriali disponibili — è compatibile con il salto fra $k = 8$ e $k = 20$, ma non spiega perché B_X resti costante fino a $k = 64$ né perché il fallback a $KB = 32$, che tiene al massimo 32 accumulatori indipendentemente da k , mostri lo stesso plateau. Le due cause non si escludono, e il terzo controllo elencato sopra le separa in modo netto. È la verifica da eseguire prima della consegna.

9.2.2 Conseguenza pratica

Se l'ipotesi regge, il rimedio è noto e non richiede di cambiare schema: basta bloccare il ciclo su j in fette di X che stiano in cache, riusando ogni fetta su tutte le righe di A prima di passare alla successiva — lo stesso principio che `cuda_warp_smem` applica sulla GPU con la shared memory (sezione 5.4). È il primo dei lavori futuri (sezione 10.3), ed è anche il motivo per cui vale la pena identificare la causa e non limitarsi a osservare che “oltre $k = 8$ va peggio”.

Figura da generare

GFLOPS in funzione di k , una curva per backend, scala logaritmica sull'asse y . Sovrapporre come linea orizzontale il tetto FP64 di tabella 6.2 (350 GFLOP/s): la distanza da quel tetto è il risultato, non il numero assoluto.

Figura 9.2: C2: confronto fra backend al variare di k .

Le tre domande a cui questa campagna deve rispondere, e che vanno affrontate esplicitamente nel commento ai dati:

1. **Quanto vale il passaggio da `cuda_naive` a `cuda_warp`?** È la misura del recupero del riuso di A in registro e della coalescenza, cioè del salto di intensità aritmetica della eq. (2.3). Ci si attende un guadagno crescente con k , perché è k il fattore di riuso perduto dalla baseline.
2. **La shared memory guadagna qualcosa?** Il roofline di sezione 6.1 prevede di no, per $k \geq 6$ in FP64: se la misura lo conferma, la conferma è il risultato. Il controllo incrociato è ripetere il confronto in FP32, dove il modello prevede un regime memory-bound e quindi un guadagno.
3. **Dove sta il crossover fra `cuda_warp_smem` e `cuda_warp_tiled`?** Il primo tiene k accumulatori per lane, il secondo ne tiene `WARP_COL_TILE` al prezzo di rileggere A : il rapporto fra i due tempi in funzione di k individua il punto in cui la pressione sui registri costa più della rilettura.

9.3.1 Sweep sul padding della shared memory

Da completare con i dati della campagna

Confronto `cuda_warp_smem` con `SMEM_PAD=1` contro `SMEM_PAD=0` a $k = 32$, dove l'analisi di sezione 5.4.1 prevede il conflitto a 32 vie. È una previsione puntuale e falsificabile: il padding deve mostrare un guadagno a $k = 32$ e nessun effetto apprezzabile a $k = 3$.

k	SMEM_PAD=0	SMEM_PAD=1	guadagno
3	—	—	—
8	—	—	—
32	—	—	— (atteso: il più alto)

Tabella 9.6: Effetto del padding sul conflitto di banchi. **[DA FARE: eseguire]**

9.3.2 Sweep sulla dimensione del blocco

Da completare con i dati della campagna

$\text{BLOCK} \in \{64, 128, 192, 256, 384, 512, 1024\}$ su `cuda_warp`, a k fisso. Serve a documentare che il default 256 non è stato scelto solo per argomento teorico (divisore di 1024, quattro blocchi per SM) ma anche misurato. I valori 192 e 384 sono i controlli: se l'argomento sull'occupancy è corretto, devono essere peggiori dei divisori di 1024.

9.4 C1 — taglia e rapporto $M:N$

Da completare con i dati della campagna

Suite `c1_size_ratio_cpu.txt` e `c1_size_ratio_gpu.txt`, i dodici punti di `??`. È la campagna esplicitamente richiesta dalla traccia e non può mancare nella consegna.

Tabella 9.7: C1: GFLOPS per passo e rapporto. Una tabella per backend.

Passo	CPU (<code>scheme_a</code>)			GPU		
	1:1	3:1	1:2	1:1	3:1	1:2
s1	—	—	—	—	—	—
s2	—	—	—	—	—	—
s3	—	—	—	—	—	—
s4	—	—	—	—	—	—

Figura da generare

GFLOPS in funzione di MN (asse x logaritmico), tre curve per i tre rapporti, un pannello per CPU e uno per GPU. Poiché il lavoro è uguale a parità di passo, ogni scostamento verticale fra le tre curve è effetto della *forma*.

Figura 9.3: C1: effetto della taglia e del rapporto $M:N$.

Da discutere nel commento: la crescita con la taglia (l'ammortamento dei costi fissi, e su GPU il raggiungimento di parallelismo sufficiente a saturare gli SM) e, separatamente, la distanza fra le tre curve, che è l'effetto della forma previsto in sezione 3.8 e in sezione 6.1 — con l'attesa che il rapporto 3:1 (M grande, molti warp) sia favorevole su GPU e che 1:2 lo sia meno.

9.5 C6 — perché la griglia è bidimensionale

Da completare con i dati della campagna

Suite `c6_grid_shape.txt`, tutte le fattorizzazioni di $P = 8, 16, 20$. È la campagna che giustifica sperimentalmente il requisito centrale della traccia; le forme $1 \times P$ e $P \times 1$ sono le due decomposizioni monodimensionali e vanno sullo stesso grafico.

Tabella 9.8: C6: $P = 20$, tutte le forme ordinate.

Forma $P_r \times P_c$	t_{bcast}	t_{reduce}	t_{total}	GFLOPS
1×20 (1D)	—	—	—	—
2×10	—	—	—	—
4×5	—	—	—	—
5×4	—	—	—	—
10×2	—	—	—	—
20×1 (1D)	—	—	—	—

Figura da generare

Due assi y sovrapposti: t_{bcast} e t_{reduce} lungo la sequenza di forme. La eq. (3.5) prevede che si muovano in direzioni *opposte* e che il minimo del totale cada in mezzo, non agli estremi.

Figura 9.4: C6: broadcast e reduce in funzione della forma della griglia.

Questa è la verifica più diretta del modello di costo: la eq. (3.6) predice, per $M = N$ e $P = 20$, un ottimo vicino a 4×5 o 5×4 , e un peggioramento marcato agli estremi. Se i dati lo confermano, la scelta di `grid_default_shape` risulta giustificata da una misura e non da un'intuizione; se non lo confermano, il modello va corretto, ed è comunque un risultato da riportare.

9.6 C4 — strong scaling

Da completare con i dati della campagna

Suite `c4_strong_scaling.txt`: $20\,000 \times 20\,000$, $P \in \{1, 2, 4, 8, 16, 20\}$, $k \in \{3, 32\}$.

Tabella 9.9: C4: speedup ed efficienza a problema fisso.

P	t	$k = 3$				$k = 32$			
		speedup	eff.	% comm.		t	speedup	eff.	% comm.
1	—	1,00	1,00	—	—	—	1,00	1,00	—
2	—	—	—	—	—	—	—	—	—
4	—	—	—	—	—	—	—	—	—
8	—	—	—	—	—	—	—	—	—
16	—	—	—	—	—	—	—	—	—
20	—	—	—	—	—	—	—	—	—

Figura da generare

A sinistra: speedup contro P , con la retta ideale come riferimento, due curve ($k = 3$ e $k = 32$). A destra: scomposizione di t_{total} in t_{bcast} , t_{local} , t_{reduce} come frazioni impilate.

Figura 9.5: C4: strong scaling e composizione del tempo.

L'attesa, dalla eq. (3.4): con $k = 3$ la comunicazione è una frazione piccola e la scalabilità resta vicina all'ideale; con $k = 32$ il volume comunicato è dieci volte tanto ed è lì che l'efficienza si degrada. Il pannello di destra deve rendere visibile *quale* delle due collettive domina, e il confronto con la legge di Amdahl applicata alla frazione non parallelizzabile misurata chiude l'analisi.

9.7 C5 — weak scaling

Da completare con i dati della campagna

Suite `c5_weak_scaling.txt`: blocco locale costante 5000×5000 , $M = 5000P_r$, $N = 5000P_c$.

Tabella 9.10: C5: tempo a lavoro per processo costante.

P	Griglia	$M \times N$	t_{total}	t_{comm}	efficienza
1	1×1	5000×5000	—	—	1,00
2	—	—	—	—	—
4	—	—	—	—	—
8	—	—	—	—	—
16	—	—	—	—	—
20	—	—	—	—	—

Poiché il lavoro per rank è costante, il tempo ideale è piatto e tutto ciò che sale è comunicazione pura: è la misura più pulita del costo delle collettive, e il confronto con la previsione della eq. (3.4) permette di stimare α e β per interpolazione. Va ricordato nel commento che su

una griglia 2D il weak scaling cambia anche il rapporto $M:N$ (sezione 3.8), quindi l'effetto misurato qui non è puramente quello di P .

9.8 C7.1 — quanto rende la singola precisione

Da completare con i dati della campagna

Suite `experiments/c7_1_precision_k.txt`: cinque backend GPU in coppie double/float, $M = N = 10\,000$, $P = 1$, BLOCK=256, WARP_COL_TILE=8. Colonna da leggere: `t_kernel_mean_s`, e da essa lo speedup della ??.

Tabella 9.11: C7.1: rapporto $S_{32/64} = t_{\text{kernel}}^{\text{fp64}}/t_{\text{kernel}}^{\text{fp32}}$ per backend e per k .

Backend	$k = 3$	$k = 6$	$k = 8$	$k = 20$	$k = 32$
<code>cuda_naive</code>	—	—	—	—	—
<code>cuda_warp</code>	—	—	—	—	—
<code>cuda_warp_smem</code>	—	—	—	—	—
<code>cuda_warp_tiled</code>	—	—	—	—	—
<code>cublas</code>	—	—	—	—	—
<i>previsione roofline</i>	≈ 2	<i>crescente con k, fino a $\sim 20\times$ a $k = 32$</i>			

È la verifica più diretta del modello di capitolo 6, ed è falsificabile in modo netto. Il modello prevede che in FP64 il tetto sia il calcolo e non dipenda da k , mentre in FP32 sia la banda e cresca linearmente con k : il rapporto fra i due deve quindi **crescere con k** . Uno speedup piatto e vicino a 2 — il rapporto fra le dimensioni degli scalari, cioè ciò che si otterrebbe se entrambe le precisioni fossero memory-bound — falsificherebbe la previsione, e andrebbe riportato come tale.

Figura da generare

$S_{32/64}$ in funzione di k , una curva per backend, con due linee orizzontali di riferimento: $S = 2$ (entrambe memory-bound) e la previsione del roofline. È il grafico che decide se il modello di capitolo 6 regge.

Figura 9.6: C7.1: guadagno della singola precisione al variare di k .

Per `cuda_warp_smem` vanno riportate anche le due colonne del piano, perché cambiano con la precisione e sono metà della spiegazione:

k	x_rows_per_tile		blocks_per_sm	
	double	float	double	float
3	—	—	—	—
8	—	—	—	—
20	—	—	—	—
32	—	—	—	—

Tabella 9.12: C7.1: piano di tiling scelto a runtime nelle due precisioni. **[DA FARE: compilare]**

9.9 C7.2 — perché: registri, spill e shared memory

Da completare con i dati della campagna

Suite `experiments/c7_2_precision_resources.txt`: sei compilazioni con `-Xptxas -v`, nessun benchmark. Estrarre dal log completo `*_ptxas*.txt` i casi $k \in \{3, 8, 20, 32\}$.

Tabella 9.13: C7.2: risorse per thread dichiarate da `ptxas` (registri a 32 bit in entrambe le precisioni).

Backend	Prec.	k	reg./thread	stack	spill st./ld.	shared statica
cuda_warp	double	32	—	—	—	—
cuda_warp	float	32	—	—	—	—
cuda_warp_smem	double	32	—	—	—	—
cuda_warp_smem	float	32	—	—	—	—
cuda_warp_tiled	double	32	—	—	—	—
cuda_warp_tiled	float	32	—	—	—	—
<i>ripetere per $k = 3, 8, 20$</i>						

L'attesa è leggibile in anticipo, ed è ciò che rende la tabella un esperimento e non un inventario: a $k = 32$ un accumulatore `double` occupa due registri a 32 bit e uno `float` uno solo, quindi il passaggio a FP32 dovrebbe dimezzare la pressione sui registri e, se a $k = 32$ in doppia precisione compare dello spill, farlo sparire. Se lo spill c'è, spiega *da solo* una parte dello speedup di tabella 9.11 che il roofline non prevedeva — ed è il motivo per cui le due metà della campagna vanno lette insieme.

Due limiti da rispettare nella lettura, che la campagna stessa dichiara: `ptxas` riporta la shared memory *statica* e non quella dinamica, quindi per `cuda_warp_smem` il dato va ricostruito dal piano runtime di tabella 9.12 come `x_rows_per_tile · (k + 1) · s` byte; e `blocks_per_sm` è una capacità teorica dichiarata dall'API dell'occupancy, non l'occupancy raggiunta, che richiederebbe un profilo con `ncu`.

9.10 Il costo dei dati: trasferimenti e preprocessing

Da completare con i dati della campagna

Nessuna esecuzione aggiuntiva: le colonne di `??` sono già in coda al CSV di ogni campagna. Compilare le due tabelle seguenti estraendole dalle righe GPU di C2 (per la dipendenza da k) e di C1 (per la dipendenza dalla taglia).

Questa sezione risponde a una domanda che la traccia permette di non mettere in T ma non di ignorare: con $k \leq 32$, quanto costa portare i dati sulla scheda rispetto a calcolarci sopra?

Tabella 9.14: Scomposizione della fase locale su GPU, secondo la eq. (5.1). Percentuali di t_{local} .

k	$t_{\text{H2D } X}$	t_{kernel}	$t_{\text{D2H } Y}$	t_{launch}	$t_{\text{local}} [\text{s}]$
3	—	—	—	—	—
8	—	—	—	—	—
32	—	—	—	—	—

Da discutere: i due trasferimenti per invocazione scalano con $n_{\text{loc}}k$ e $m_{\text{loc}}k$, mentre l’overhead del runtime non scala affatto — quindi il loro peso relativo deve muoversi in direzioni opposte al crescere della taglia, ed è esattamente ciò che la scomposizione permette di verificare invece di asserire. Se a $k = 3$ i trasferimenti pesano quanto il kernel o più, quello è il risultato: dice che a k piccolo il problema non è il calcolo, ed è l’argomento più forte per un MPI CUDA-aware (sezione 10.3).

Tabella 9.15: Preprocessing e banda dei trasferimenti, per una configurazione rappresentativa.

Voce	Valore	Nota
t_{init}	—	costo fisso, indipendente dalla taglia
t_{setup}	—	cresce con la taglia
t_{alloc}	—	il trasferimento tolto dal cammino misurato
$t_{\text{H2D } A}$	—	da confrontare con il picco del bus
t_{setup}	—	messaggi piccoli: attesa banda molto inferiore
bw_h2d_A_gbs	—	generazione, esclusa da T
$\text{bw_h2d_X_gbs} / \text{bw_d2h_Y_gbs}$	—	0 con <code>-a-mode local</code>
$t_{\text{gen } A}$	—	0 con <code>X_LAYOUT=row</code>
$t_{\text{distrib } A}$	—	
t_{prep}	—	
$t_{\text{convert } X}$	—	
t_{prep}	—	

Il confronto fra `bw_h2d_A_gbs` e le due bande per invocazione è istruttivo di per sé: A è un unico trasferimento da gigabyte e dovrebbe avvicinarsi al picco del bus, mentre X e Y sono messaggi da qualche decina o centinaio di kilobyte, dominati dalla latenza. È la stessa distinzione fra α e β della eq. (3.4), su un altro collegamento.

9.11 Sintesi

Da completare con i dati della campagna

Completare a raccolta ultimata con: la tabella riassuntiva (backend $\times k$, migliore configurazione, GFLOPS, % del tetto roofline pertinente); il confronto fra la previsione di sezione 6.1 e la misura, con lo scarto; l’esito dei tre punti elencati in sezione 9.3.

Capitolo 10

Conclusioni

10.1 Che cosa è stato realizzato

Il progetto implementa il prodotto $Y \leftarrow AX$ per matrice densa e multivettore stretto, distribuito su una griglia di processi bidimensionale con MPI e accelerato all'interno del singolo processo con CUDA. Il codice è in C11 per la parte host e distribuita, in CUDA C++ per i kernel di device, e copre tutti i requisiti della traccia elencati in ??.

Le scelte che hanno avuto l'effetto maggiore, e che sono anche quelle su cui poggia l'analisi:

- **lo schema A** ($i \rightarrow j \rightarrow c$), che legge ogni elemento di A una volta sola e lo riusa nei k accumulatori in registro, portando l'intensità aritmetica da $2/s$ a $2k/s$ FLOP/B — il massimo ottenibile leggendo A una volta;
- **la griglia 2D**, che riduce il volume comunicato da $\Theta(kM)$ o $\Theta(kN)$ delle decomposizioni monodimensionali a $\Theta(k\sqrt{MN}/\sqrt{P})$;
- **il ciclo di vita a tre fasi** del kernel locale, che sposta fuori dal cronometro tutto ciò che riguarda A — inclusa la copia H2D, che altrimenti misurerebbe il PCIe invece della GPU — rendendo la prescrizione della traccia una proprietà strutturale e non una sottrazione a posteriori;
- **una variabile per volta** fra le varianti di kernel, che è la condizione perché ogni differenza misurata sia attribuibile;
- **la strumentazione completa dei tempi esclusi (??)**: la traccia permette di non metterli in T , e il progetto li misura comunque, scomposti per fase. La fase locale su GPU è separata nei suoi quattro addendi (eq. (5.1)), il preprocessing del backend nelle sue tre voci, e i trasferimenti riportano anche i byte, così che un tempo di copia si possa giudicare e non solo constatare.

10.2 Che cosa dicono i dati finora

Da completare con i dati della campagna

Questa sezione va riscritta a raccolta ultimata. Al momento l'unica campagna disponibile è C3, e il suo risultato principale è riportato qui sotto.

Il nucleo di CPU raggiunge 10,9 GFLOP/s a $k = 8$ su $10\,000 \times 10\,000$ in doppia precisione a rank singolo, e presenta una discontinuità netta fra $k = 8$ e $k = 20$: il costo per elemento di A passa da 1,47 ns a 7,68 ns, mentre il lavoro cresce solo di $2,5\times$. L'analisi di sezione 9.2 attribuisce il fenomeno alla rilettura di X , che oltre una certa soglia esce dalla cache privata del core, e propone tre controlli per falsificare l'ipotesi. La firma a sostegno è un plateau

di banda effettiva su X intorno a da 17 GB/s a 21 GB/s, costante su cinque valori di k che coprono un fattore 3,2.

Il modello roofline prevede inoltre che sulla GPU, in doppia precisione, il nucleo sia compute-bound per ogni $k \geq 6$, con tetto a 350 GFLOP/s indipendente da k , mentre in singola precisione resti memory-bound per tutti i k della traccia con tetto proporzionale a k . È una previsione forte e falsificabile, e le campagne C1, C2 e soprattutto C7 la mettono alla prova: quest'ultima ne isola la conseguenza più netta — il guadagno della singola precisione deve crescere con k — e la accompagna con i dati di `ptxas` che dicono se una parte del guadagno venga invece dalla pressione sui registri.

10.3 Lavori futuri

Blocking del ciclo su j sul nucleo di CPU. È la conseguenza diretta dell'analisi di sezione 9.2: percorrere X a fette che stiano nella cache privata, riusando ogni fetta su tutte le righe di A prima di passare alla successiva. Non cambia lo schema, cambia solo l'ordine di visita, ed è lo stesso principio che `cuda_warp_smem` applica sulla GPU con la shared memory. Se l'ipotesi sulla soglia di cache regge, è l'intervento con il rapporto guadagno/costo migliore dell'intero progetto per $k \geq 20$.

Parallelizzazione OpenMP interna al processo. La traccia ammette OpenMP *oppure* CUDA e il progetto ha scelto CUDA. Un backend OpenMP dietro la stessa interfaccia `local_gemm` permetterebbe di usare i circa venti core del nodo anche quando la GPU è occupata, e di misurare il regime ibrido MPI+OpenMP con lo stesso cronometro degli altri.

MPI CUDA-aware. L'interfaccia `local_gemm` è stata progettata agnostica rispetto alla posizione della memoria proprio per consentirlo: passando puntatori device direttamente a `MPI_Bcast` e `MPI_Reduce` si eliminerebbero H2D di X e D2H di Y dal cammino di ogni invocazione, senza riscrivere né `mpi_matmul` né i kernel. Il primo passo è verificare il supporto sul nodo di calcolo con `ompi_info | grep cuda`; il secondo è già fatto, perché la scomposizione di sezione 9.10 dice esattamente quanto ci sarebbe da guadagnare — $t_{H2D} X$ e $t_{D2H} Y$ misurati sono il tetto di quel guadagno, e senza quei due numeri l'intervento sarebbe una scommessa.

MPS per la condivisione della GPU. Con MPS attivo i contesti di più rank non si alternerebbero sulla scheda (sezione 5.7), e i numeri multi-rank su GPU tornerebbero interpretabili come tempo di kernel invece che come throughput aggregato. È anche il modo per replicare, con analisi esplicita della serializzazione, i benchmark multi-griglia su GPU — un punto che i lavori di confronto lasciano scoperto.

Profilazione con `ncu`. Le ipotesi formulate nei capitoli precedenti hanno tutte un contatore hardware che le decide: occupancy e banda L2 per `cuda_warp_smem`, conflitti di banco per lo sweep su `SMEM_PAD`, registri per warp per `cuda_warp_tiled`, kernel interno selezionato per `cublas`. È il passo che trasforma le spiegazioni proposte in spiegazioni verificate.

Appendice A

Interfaccia a riga di comando e formato dei dati

A.1 Opzioni del driver

Opzione	Default	Significato
-M <int>	1024	righe di A
-N <int>	1024	colonne di A (e righe di X)
-k <int>	8	colonne del multivettore
-pr <int>	più quadrata	righe della griglia di processi
-pc <int>	più quadrata	colonne della griglia di processi
-reps <int>	10	repetition cronometrate
-warmup <int>	2	repetition di riscaldamento, non cronometrate
-seed <u64>	fisso	seme del generatore
-a-mode <mode>	local	local: ogni processo genera il proprio blocco; global: il rank 0 materializza A e la distribuisce
-x-mode <mode>	local	come sopra per X , sulla sola riga 0 della griglia
-check	off	valida contro il riferimento seriale
-csv	off	stampa una riga CSV invece del report leggibile
-csv-header	—	stampa l'intestazione CSV ed esce
-csv-raw-file <path>	—	appende una riga per repetition a un CSV “raw”
-h, -help	—	elenco delle opzioni

Tabella A.1: Opzioni di bin/matmul_mpi.

Le quattro combinazioni di `-a-mode` e `-x-mode` sono tutte valide e, a parità di seed, producono gli stessi elementi globali.

`-help` stampa anche il layout di X della build corrente (`X_LAYOUT=row|column`): non è un'opzione di riga di comando, ma è la prima cosa da verificare quando due binari danno tempi diversi sullo stesso problema.

A.2 Colonne del CSV

L'intestazione è emessa dal binario stesso (`-csv-header`), quindi non può divergere dalle colonne effettivamente scritte. È la ragione per cui tutti i CSV di una campagna si possono concatenare senza rischio.

Gruppo di colonne	Contenuto
kernel, scalar, a_mode, x_mode	identificazione della build e della modalità; kernel include i parametri che cambiano il codice generato
M, N, k, P, pr, pc, reps	configurazione del problema
t_bcast_*, t_local_*, t_reduce_*	tempi delle tre fasi, media e deviazione standard
t_total_*	tempo end-to-end reale: media, mediana, minimo, deviazione, CV%
t_official_*	metrica ufficiale della ??, stesse statistiche
t_kernel_*	solo kernel, misurato con cudaEvent ; vale -1 sui backend di CPU
t_transfer_runtime_overhead_*	$t_{\text{local}} - t_{\text{kernel}}$: trasferimenti e overhead del runtime host
t_setup_s	preprocessing del backend, fuori da tutte le metriche
gflops, gflops_compute, gflops_kernel	le tre metriche di ??
blocks_per_sm, x_rows_per_tile	piano di esecuzione scelto a runtime dal backend; -1 dove il concetto non esiste (sezione 5.4.2)
rel_err	errore relativo contro l'oracolo, oppure -1 senza -check
<i>da qui in poi: voci ESCLUSE dalla metrica ufficiale (??), in coda così che le colonne storiche non si spostino</i>	
t_setup_device_init_s, t_setup_device_alloc_s, t_setup_h2d_A_s, bw_h2d_A_gbs	scomposizione del preprocessing del backend e banda della copia di A
t_h2d_X_*, bw_h2d_X_gbs	H2D di X per invocazione
t_d2h_Y_*, bw_d2h_Y_gbs	D2H di Y per invocazione
t_launch_overhead_*	residuo della eq. (5.1)
t_prep_*	le sei fasi del preprocessing del driver e il loro totale
t_io_csv_raw_s	unico I/O del driver
x_layout	row o column (sezione 5.3.1)
t_prep_convert_X_s	costo della conversione di layout, 0 in row

Tabella A.2: Struttura dell'intestazione CSV.

Il CSV *raw* ha un'altra intestazione — una riga per repetition, senza statistiche, con **t_h2d_X_s**, **t_d2h_Y_s**, **t_launch_overhead_s** e **x_layout** — e va unito separatamente da quello aggregato. Il driver rifiuta di appendere a un file la cui intestazione non coincide con la propria: un CSV misto fra due build sarebbe ambiguo senza dare alcun errore.

A.3 Opzioni di build

Alcune scelte non sono opzioni di riga di comando ma di *compilazione*, perché cambiano il codice generato: metterle a runtime introdurrebbe rami nel percorso caldo e renderebbe impossibile sapere, guardando un binario, che cosa stia misurando. Ognuna entra nel nome del binario e in **kernel_name()** (??).

Variabile	Default	Effetto
KERNEL	scheme_a	implementazione di <code>local_gemm</code> ; <code>.cu</code> ha la precedenza su <code>.c</code> e attiva <code>nvcc</code>
PREC	double	tipo scalare (<code>-DUSE_FLOAT</code>)
FORCE_GENERIC_K	0	disattiva le specializzazioni su k di <code>scheme_a</code>
TEST_A_PADDING	0	impone $lda = n_{loc} + n$
BLOCK	256	thread per blocco dei backend CUDA
SMEM_PAD	1	scalari di padding per riga del tile (sezione 5.4.1)
TILE_GRANULARITY	32	arrotondamento delle righe per tile
SMEM_BUDGET_BYTES	derivato	forza il budget di shared memory invece di derivarlo dall'occupancy
WARP_COL_TILE	8	colonne per warp in <code>cuda_warp_tiled</code>
X_LAYOUT	row	<code>column</code> rende X compatta per colonne; solo <code>cuda_warp</code> (sezione 5.3.1)
NVCC_ARCH	sm_75	architettura di destinazione

Tabella A.3: Variabili del Makefile.

Appendice B

Riproduzione dei risultati

B.1 Build

```
make                                # bin/matmul_mpi (double, scheme_a)
make FORCE_GENERIC_K=1              # bin/matmul_mpi-generic
make PREC=float                    # bin/matmul_mpi-float
make KERNEL=cuda_naive             # richiede nvcc: solo sul server
make KERNEL=cuda_warp
make KERNEL=cuda_warp X_LAYOUT=column # X compatta per colonne (solo cuda_warp)
make KERNEL=cuda_warp_smem
make KERNEL=cuda_warp_smem SMEM_PAD=0
make KERNEL=cuda_warp_smem TILE_GRANULARITY=8
make KERNEL=cuda_warp_smem SMEM_BUDGET_BYTES=16384
make KERNEL=cuda_warp_tiled
make KERNEL=cuda_warp_tiled WARP_COL_TILE=4
make KERNEL=cublas                 # aggiunge -lcublas da solo
```

Listato B.1: Compilazione delle configurazioni usate nelle campagne.

La riga finale di ogni `make` ricorda quale binario ha prodotto e con quali variabili, budget di shared memory incluso (derivato quando non è forzato).

Ogni comando stampa in chiusura quale binario ha prodotto e con quali variabili. Su una macchina priva di `nvcc` le build CUDA si fermano con un messaggio esplicito e tutto il resto continua a funzionare.

B.2 Validazione

Da eseguire su taglie piccole, prima delle campagne e per ogni combinazione di backend e precisione che si intende misurare.

```
make test                          # funzioni indice, senza MPI
make check-cxx                     # kernel.h e util.h includibili da C++
make check                         # prodotto distribuito contro oracolo, double
make PREC=float check              # stessa validazione in singola precisione
make check-padding                 # lda = n_loc + 8
make KERNEL=cuda_warp check
make KERNEL=cuda_warp_smem check
make KERNEL=cublas check
```

B.3 Esecuzione singola

```
# CPU, 4 rank su griglia 2x2
mpirun -np 4 --bind-to core --map-by core \
  ./bin/matmul_mpi -M 8000 -N 8000 -k 8 --pr 2 --pc 2 --reps 10

# GPU, rank singolo: la configurazione con cui si caratterizza un kernel
mpirun -np 1 --bind-to core --map-by core ./bin/matmul_mpi-cuda_warp \
  -M 20000 -N 20000 -k 8 --pr 1 --pc 1 --a-mode local --warmup 2 --reps 10
```

Il binding non è facoltativo sulle esecuzioni multi-rank: senza, le misure sono rumore.

B.4 Campagne

```
./script/run_cuda_experiments.sh --suite experiments/c3_cpu_baseline.txt
./script/run_cuda_experiments.sh --suite experiments/c2_k_sweep.txt
./script/run_cuda_experiments.sh --suite experiments/c1_size_ratio_cpu.txt
./script/run_cuda_experiments.sh --suite experiments/c1_size_ratio_gpu.txt
./script/run_cuda_experiments.sh --suite experiments/c1_size_ratio_mpi20.txt
./script/run_cuda_experiments.sh --suite experiments/c6_grid_shape.txt
./script/run_cuda_experiments.sh --suite experiments/c4_strong_scaling.txt
./script/run_cuda_experiments.sh --suite experiments/c5_weak_scaling.txt
./script/run_cuda_experiments.sh --suite experiments/c7_1_precision_k.txt
./script/run_cuda_experiments.sh --suite experiments/c7_2_precision_resources.txt
```

C7.2 non esegue benchmark: ricompila con `-Xptxas -v` e raccoglie i log delle risorse. I casi k vanno selezionati dal log completo `*_ptxas*.txt`, non dal solo `*_registers.txt`, che omette i nomi delle funzioni e quindi non permette di attribuire i registri al k giusto.

Prima di rilanciare una campagna

Rilanciare sovrascrive i CSV della campagna. Archiviare la cartella:

```
mv results/c4_strong_scaling results/c4_strong_scaling.$(date +%F_%H%M)
```

B.5 Unione dei CSV di una campagna

Tutti i CSV aggregati di una cartella condividono l'intestazione:

```
find results/c1_size_ratio_cpu -name '*.csv' \
  ! -name '*_raw.csv' ! -name '*_failures.csv' \
  | sort | xargs awk 'FNR==1 && NR!=1 {next} {print}' \
  > results/c1_size_ratio_cpu/merged.csv
```

I file `*_raw.csv` hanno un'altra intestazione e vanno uniti separatamente.

B.6 Note operative sul server

- **Nodo di calcolo, non frontend.** Il nodo di login non ha accesso diretto alla GPU: `nvcc -version`, `mpi_info | grep cuda`, `deviceQuery` e `nvidia-smi` vanno eseguiti sui nodi di calcolo attraverso lo scheduler, e così tutti i benchmark.

- **Trasporto MPI.** Il runner usa `btl=self,sm` per default. Il componente `sm` non esiste in Open MPI 3.x/4.x, dove si chiama `vader`: se le esecuzioni multi-rank falliscono con un errore di BTL, verificare con `ompi_info -param btl all` e aggiungere `-btl self,vader` sulla riga di comando, che ha la precedenza sul `.conf`.
- **bash ≥ 4.4 .** Il runner espande array potenzialmente vuoti sotto `set -u`: su bash 3.2 (macOS) si ferma con `unbound variable`. Sul server Linux non si presenta.
- **Memoria della GPU.** Un blocco locale $40\,000^2$ in doppia precisione richiede 12,8 GiB contro i 15,5 GiB della scheda: il programma si ferma prima di allocare, indicando quanta memoria serve e quanta è libera.

B.7 Compilazione di questa relazione

```
cd report && ./compila.sh    # oppure: latexmk -pdf main.tex
```

Per la versione di consegna, commentare la riga `\todovisibiletrue` in `preambolo.tex`: i marcatori dei dati mancanti spariscono dal PDF e restano nel sorgente.